Αναγνώριση Προτύπων - Εργαστήριο 1

Αναγνώριση φωνής με Κρυφά Μαρκοβιανά Μοντέλα και Αναδρομικά Νευρωνικά Δίκτυα

Κόρδας Νικόλαος (03121032)
Κριθαρίδης Κωνσταντίνος (03121045)

November 13, 2025

Περιγραφή

Ο μηχανισμός παραγωγής φωνής

Φωνητική οδός (vocal tract): Πρόκειται για τη σειρά των οργάνων στον ανθρώπινο λαιμό και το κεφάλι που συνδέονται με την παραγωγή ήχου, από το λάρυγγα μέχρι τα χείλη και τη μύτη. Ο αρχικός ήχος παράγεται από την δόνηση των φωνητικών χορδών στον λάρυγγα. Ο ήχος ταξιδεύει μέσω της φωνητικής οδού. Το φάσμα της φωνής διαμορφώνεται από τη συχνοτική επιλεκτικότητα της φωνητικής οδού.

Formants: Οι συχνότητες συντονισμού του φωνητικού σωλήνα ονομάζονται formants.
Όταν μετράμε τα formants για τα φωνήεντα α, ου, ι, στην πραγματικότητα μετράμε τις συχνότητες συντονισμού του φωνητικού σωλήνα για το καθένα τους. Δεδομένου ότι ο φωνητικός σωλήνας των ανδρών είναι γενικά μεγαλύτερος από αυτόν των γυναικών, τα formants του ομιλητή 1 αναμένονται σε χαμηλότερες συχνότητες.

Pitch: Είναι η θεμελιώδης συχνότητα ταλάντωσης των φωνητικών χορδών κατά την παραγωγή του ήχου. Συνήθως, είναι χαμηλότερη στους άνδρες ενώ επηρεάζεται από το συναισθηματισμό, το τόνο και την προσωδία της ομιλίας. [**?**]

Βήμα 1

Χρησιμοποιούμε το praat για να αναλύσουμε τα σήματα ήχου μας και να εξάγουμε τις πληροφορίες που ζητούνται στην εκφώνηση.

Αρχείο onetwothree1.wav

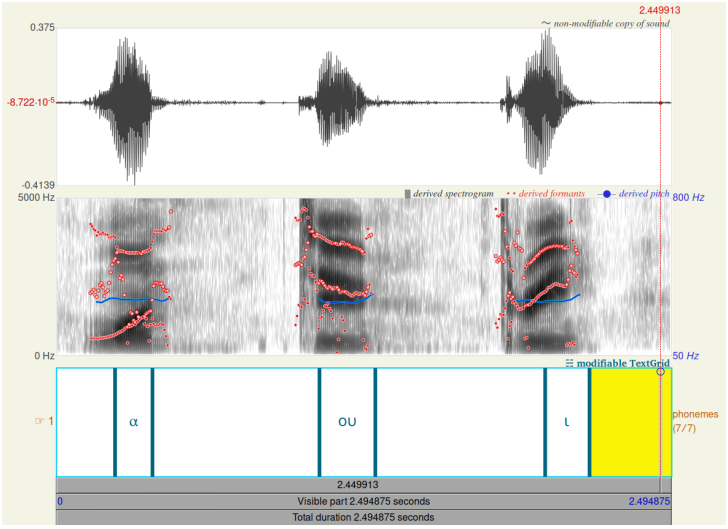
Ανοίγουμε το αρχείο στο praat για να κάνουμε annotations στα φωνήματα που μας ενδιαφέρουν. Έτσι, ορίζουμε ταυτόχρονα τις περιοχές όπου θα εξάγουμε το μέσο pitch. Πιο συγκεκριμένα, για αυτόν τον ομιλητή έχουμε ότι (από τα αρχεία που παράγει το praat):

- **Pitch (α):** 134.30091712757704 Hz (mean pitch in SELECTION)
- **Pitch (ου):** 130.34836851658105 Hz (mean pitch in SELECTION)
- **Pitch (ι):** 133.11983451151906 Hz (mean pitch in SELECTION)

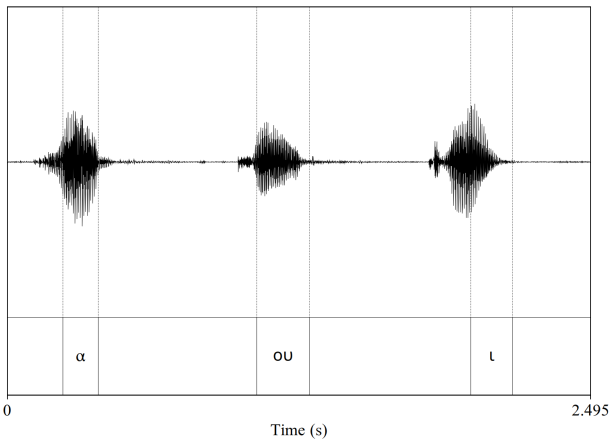
Αντίστοιχα, από το αρχείο για τα formants:

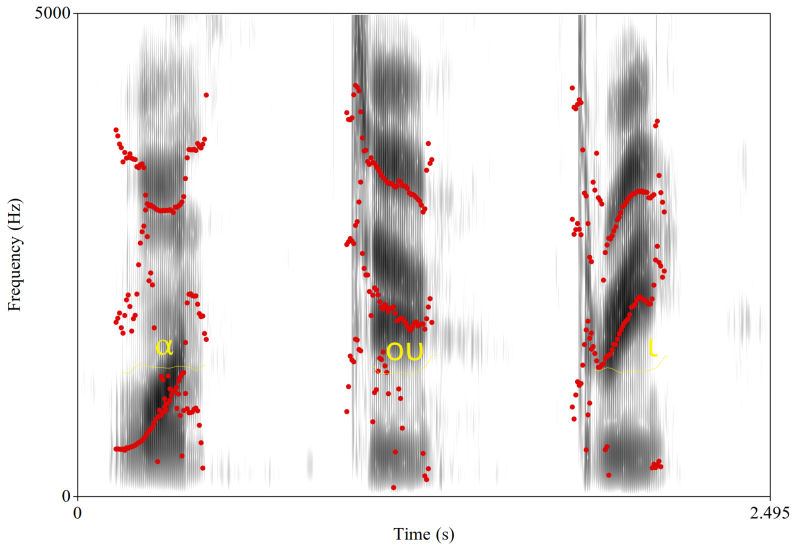
- **Φώνημα α:**
  - **F1_Hz:** 639.117504
  - **F2_Hz:** 3014.697815
  - **F3_Hz:** 3787.044970
- **Φώνημα ου:**
  - **F1_Hz:** 507.160260
  - **F2_Hz:** 2086.948531
  - **F3_Hz:** 3522.142285
- **Φώνημα ι:**
  - **F1_Hz:** 326.202470
  - **F2_Hz:** 2250.289657
  - **F3_Hz:** 3421.036991

Παρακάτω δίνουμε ένα στιγμιότυπο της διαδικασίας της επεξεργασίας και εξαγωγής των πληροφοριών:



Τέλος σχεδιάσαμε τα παρακάτω διαγράμματα που οπτικοποιούν την εργασία μας (1ο) και τις μετρήσεις μας (2ο)





Αρχείο onetwothree8.wav

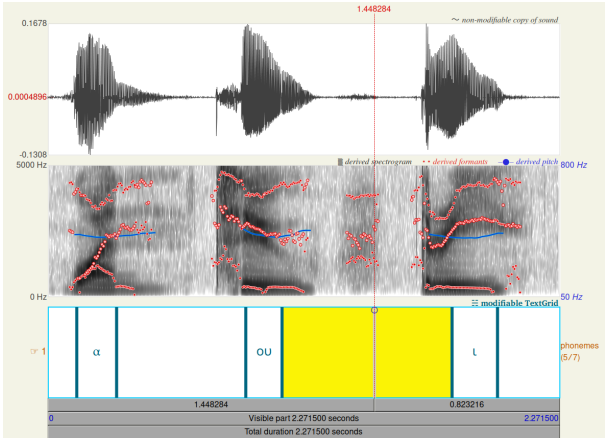
Αντίστοιχα με το προηγούμενο αρχείο οι μετρήσεις μας για το pitch κάθε φωνήματος είναι:

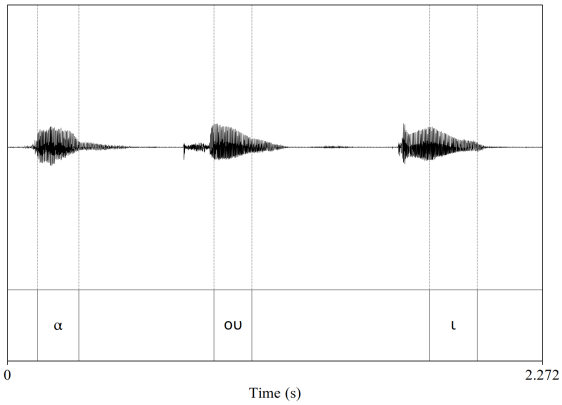
- **Pitch (α):** 178.90096253179337 Hz (mean pitch in SELECTION)
- **Pitch (ου):** 183.19462142657264 Hz (mean pitch in SELECTION)
- **Pitch (ι):** 176.88367983435367 Hz (mean pitch in SELECTION)

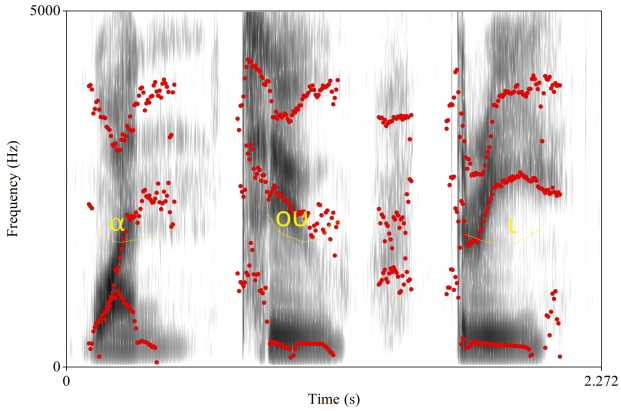
Για τα formants:

- **Φώνημα α:**
  - **F1_Hz:** 674.367922
  - **F2_Hz:** 703.507187
  - **F3_Hz:** 4112.994619
- **Φώνημα ου:**
  - **F1_Hz:** 376.486131
  - **F2_Hz:** 2801.849314
  - **F3_Hz:** 3860.898207
- **Φώνημα ι:**
  - **F1_Hz:** 345.351842
  - **F2_Hz:** 2549.549245
  - **F3_Hz:** 3396.739966

Τα αντίστοιχα διαγράμματα είναι:







Συμπεράσματα

Βλέπουμε πως οι μετρήσεις μας συμφωνούν απόλυτα με τις θεωρητικές προσδοκίες που αναφέρθηκαν στην περιγραφή παραπάνω. Πιο συγκεκριμένα, τα pitch (μέσες τιμές) της γυναίκας (αρχείο onetwothree8.wav) βρίσκονται σε υψηλότερες συχνότητες από τις αντίστοιχες του άντρα.

Επιπλέον, κατά κανόνα τα formants του άντρα βρίσκονται σε χαμηλότερες συχνότητες αντανακλώντας τα διαφορετικά φυσικά χαρακτηριστικά της φωνητικής του οδού (είναι μεγαλύτερη).

Βήμα 2

Στο βήμα αυτό ξεκινάει ο κώδικάς μας σε Python 3.
Ξεκινάμε γράφοντας την συνάρτηση data_parser που σκανάρει τον φάκελο `pr_lab2_data / digits` και επιστρέφει τρεις λίστες που περιέχουν τα δεδομένα των αρχείων wav, το id των ομιλητών και το ψηφίο που εκφωνούν, αντίστοιχα.
Τα δεδομένα του αρχείου wav τα φορτώνουμε με τη συνάρτηση load της βιβλιοθήκης librosa, τα ids των ομιλητών με την μέθοδο isdigit() των strings, και τα ψηφία με την μέθοδο isalpha() στο τμήμα του filename πριν την κατάληξη.

```
def data_parser(dir = 'pr_lab2_data/digits'):
    p = Path(dir)

    rosa_wavs = []
    speakers = []
    digits = []

    files = list(p.glob('*.wav')) # e.g. PosixPath('pr_lab2_data/digits/seven11.wav')

    for file in files:
        data, _ = librosa.load(file, sr=None)
        rosa_wavs.append(data)

        filename = file.parts[-1] # e.g. seven11.wav
        speakers.append(''.join([d for d in filename if d.isdigit()]))

        digits.append(''.join([char for char in filename.split('.')[-2] if char.isalpha()]))

    return rosa_wavs, speakers, digits
```

Στην συνάρτηση main καλούμε την συνάρτηση data_parser που μόλις γράψαμε ως εξής:

```
    # Step 2
    wavs, speakers, digits = data_parser()
    analyze_digits_statistics(digits)
```

Στο τέλος, όπως φαίνεται στον κώδικα παραπάνω, κάνουμε μία ανάλυση των στατιστικών των ψηφίων.

```
============================================================
ΣΤΑΤΙΣΤΙΚΑ ΨΗΦΙΩΝ ΑΠΟ ΤΑ ΔΕΔΟΜΕΝΑ
============================================================
Ψηφίο 'eight':  14 φορές ( 10.53%)
Ψηφίο 'five':  15 φορές ( 11.28%)
Ψηφίο 'four':  15 φορές ( 11.28%)
Ψηφίο 'nine':  15 φορές ( 11.28%)
Ψηφίο 'one':  15 φορές ( 11.28%)
Ψηφίο 'seven':  15 φορές ( 11.28%)
Ψηφίο 'six':  14 φορές ( 10.53%)
Ψηφίο 'three':  15 φορές ( 11.28%)
Ψηφίο 'two':  15 φορές ( 11.28%)
============================================================
Σύνολο δειγμάτων: 133
Μοναδικά ψηφία: 9
Πιο συχνό ψηφίο: seven (15 φορές)
============================================================
```

Αυτό το κάνουμε για να εξετάσουμε αν το dataset είναι ισορροπημένο ή όχι και αν όχι προς τα πού είναι biased. Όπως βλέπουμε, όμως, το dataset είναι ισορροπημένο. Έτσι, το accuracy θα είναι μία καλή μετρική για την αξιολόγηση των ταξινομητών που θα εκπαιδεύσουμε στη συνέχεια της εργασίας.

Βήμα 3

Στο βήμα αυτό εξάγουμε τα Mel-frequency cepstral coefficients (MFCCs) των σημάτων. Οι συντελεστές αυτοί προκύπτουν από Discrete Cosine Transform εφαρμοσμένο στον λογάριθμο της φασματικής πυκνότητας ισχύος ειλημμένη στην κλίμακα mel ώστε ισαπέχοντα διαστήματα συχνοτήτων να ακούγονται εξίσου μακριά στο ανθρώπινο αυτί. Η διαφορά στην κλίμακα είναι που την διαφοροποιεί από το κανονικό cepstrum, του οποίο η διαδικασία παριστάνεται και γραφικά στο παρακάτω διάγραμμα του [?].

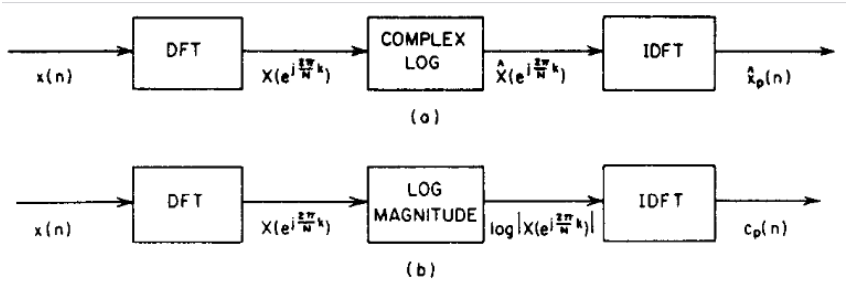


Fig. 7.7 Practical implementations of systems for obtaining (a) the complex cepstrum; and (b) the cepstrum.

Χρησιμοποιούμε παράθυρο μήκους 25ms, βήμα 10ms και εξάγουμε ως χαρακτηριστικά τα 13 MFCCs ανά αρχείο. Επιπλέον, υπολογίζουμε την πρώτη και δεύτερη διακριτή παράγωγο των MFCCs κατά την μετακίνηση του παραθύρου λήψης τους. Η βιβλιοθήκη librosa έχει συναρτήσεις για όλα αυτά τα ζητούμενα, οπότε η τελική συνάρτηση και η κλήση της στη main φαίνονται παρακάτω:

```
def extract_features(wavs):
    mfccs = []
    win_len = 25 * Fs // 1000
    step = 10 * Fs // 1000
    n_features = 13
    deltas = []
    delta_deltas = []

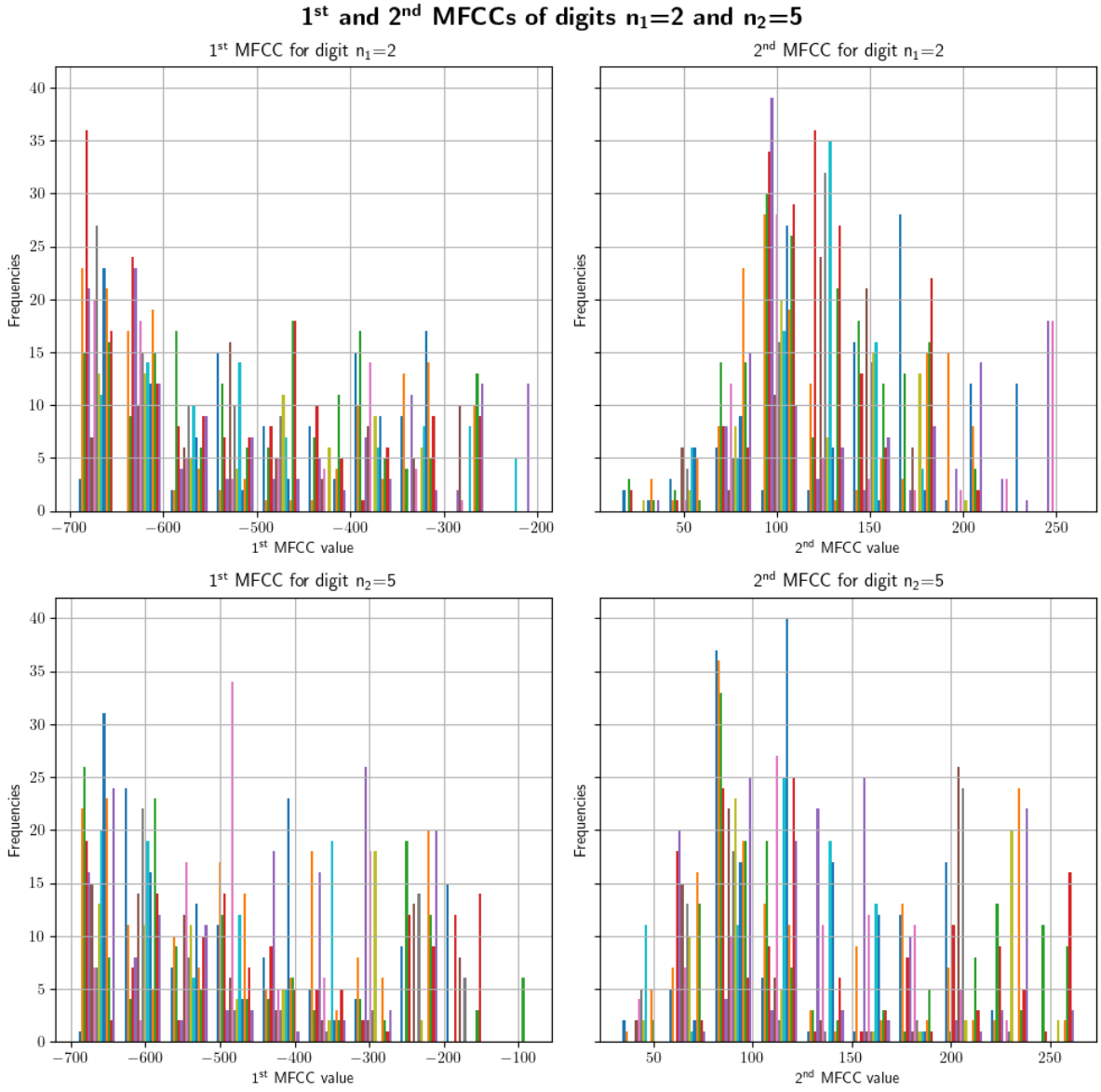
    for wav in wavs:
        mfcc = librosa.feature.mfcc(y=wav, sr=Fs, n_mfcc=n_features, hop_length
   =step, win_length=win_len)
        mfccs.append(mfcc)
        deltas.append(librosa.feature.delta(mfcc, order=1))
        delta_deltas.append(librosa.feature.delta(mfcc, order=2))

    return mfccs, deltas, delta_deltas
```

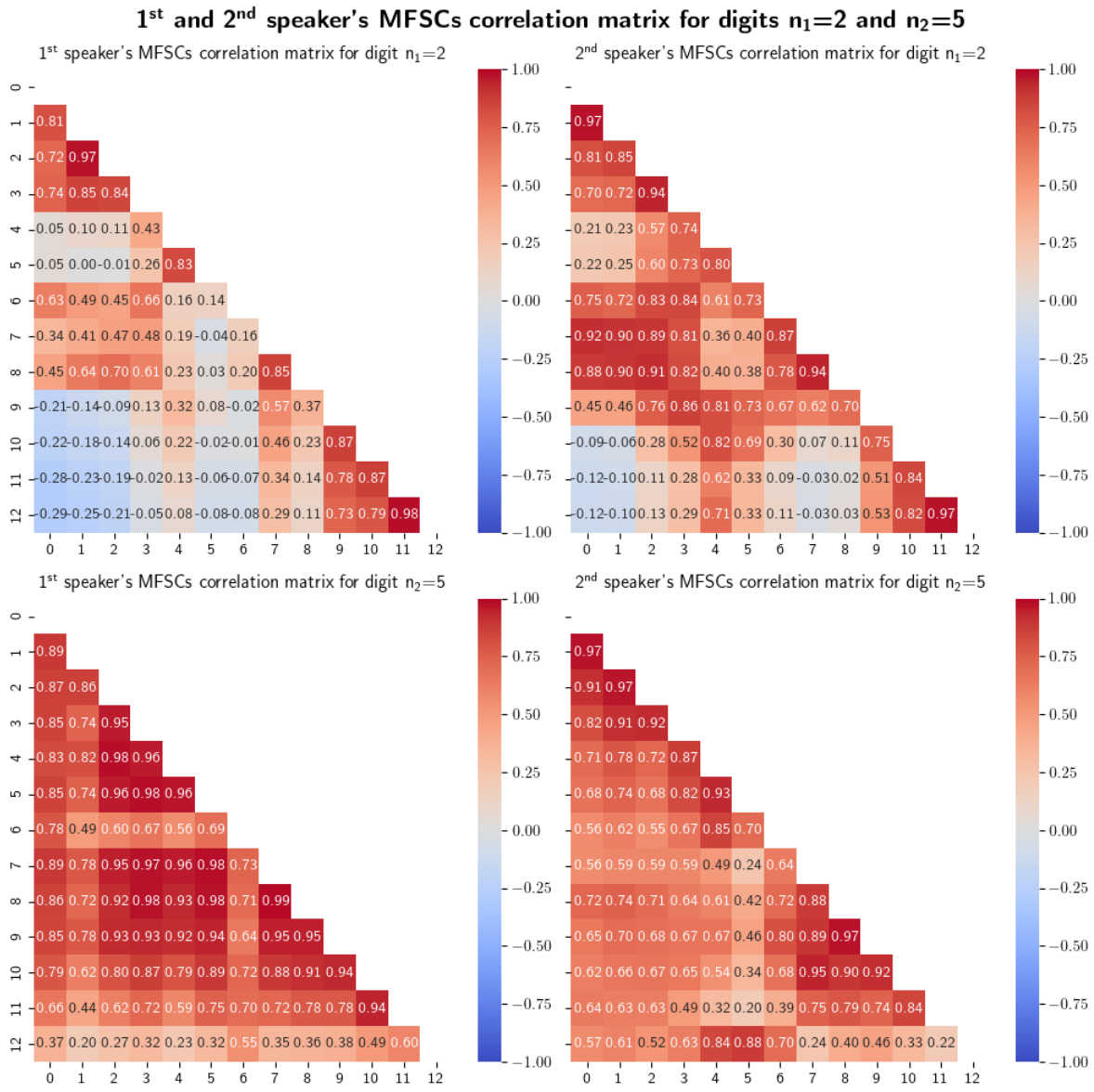
```
    # Step 3
    mfccs, deltas, delta_deltas = extract_features(wavs)
```

Βήμα 4

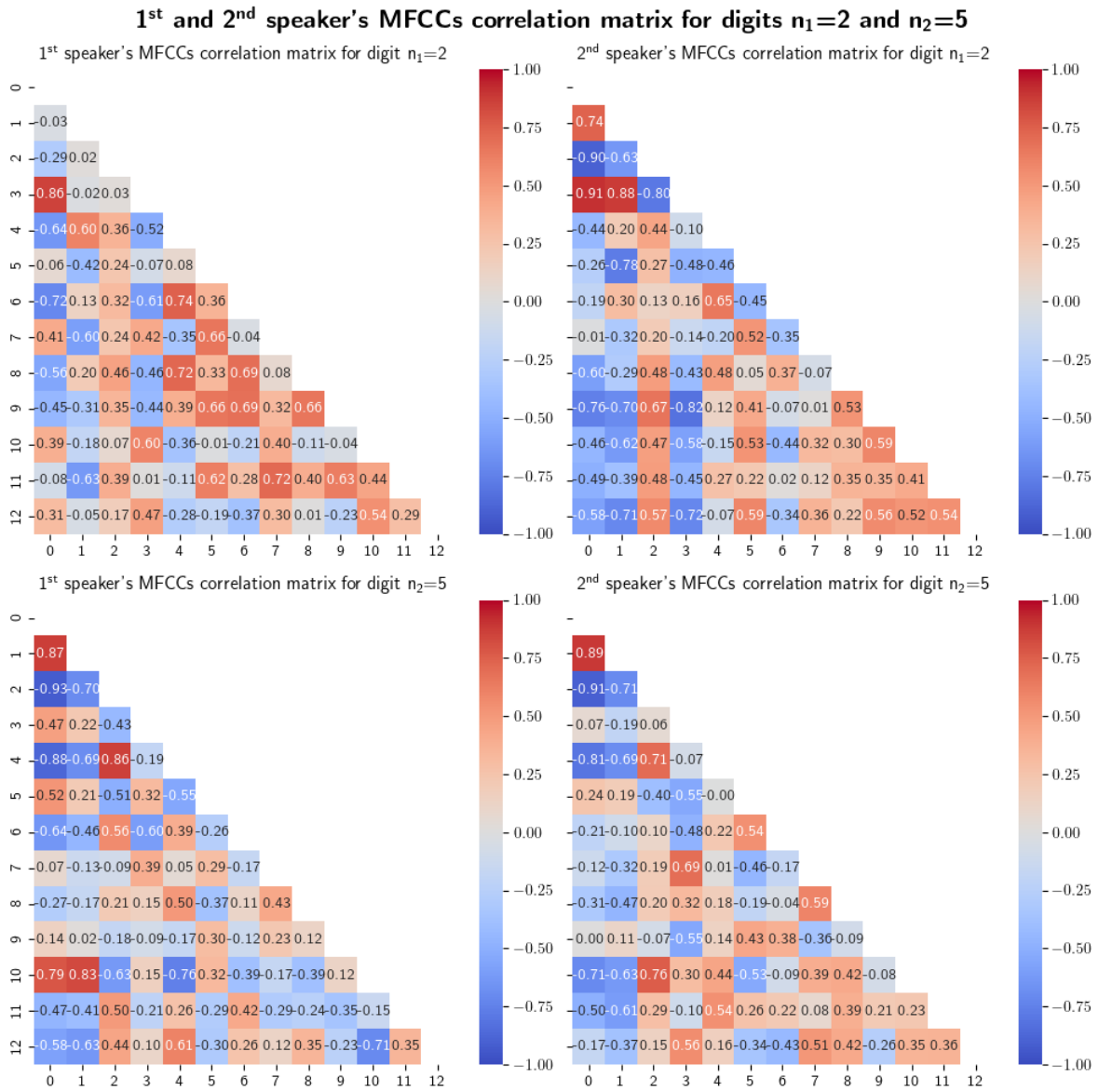
Αρχικά, δημιουργούμε ιστογράμματα για το 1ο και 2ο MFCC για όλες τις εκφωνήσεις των ψηφίων $n_1 = 2$ και $n_2 = 5$.



Αυτό που παρατηρούμε είναι ότι τα MFCC για $n_1 = 2$ δεν μοιάζουν πολύ μεταξύ τους στο ιστόγραμμα. Αυτό επιβεβαιώνεται και στο επόμενο γράφημα που αναπαριστά τη συσχέτιση μεταξύ τους, καθώς βλέπουμε ότι έχει τιμή 0,03 (γκρι χρώμα), για τον πρώτο ομιλητή. Από την άλλη πλευρά δεν παρατηρούμε το ίδιο για τον δεύτερο ομιλητή, πράγμα μη ευνοϊκό για το consistency της διαχωρισιμότητας των ψηφίων ανάμεσα στους ομιλητές. Αυτή τη παρατήρηση έρχονται να επιβεβαιώσουν και τα ιστογράμματα για $n_2 = 5$. Σε αυτά βλέπουμε ότι η μορφή τους μοιάζει αλλά και η συσχέτισή τους είναι υψηλή και για τους 2 ομιλητές (κοντά στο 0,85).
Επιπλέον, όσον αφορά την ικανότητα των MFCC να διαχωρίσουν τα διαφορετικά ψηφία, αυτό που μπορούμε να πούμε διαφάζοντας κατακόρυφα το διάγραμμα είναι ότι τα ιστογράμματα μοιάζουν μεν, δεν είναι πανομοιότυπα δε. Οπότε τα MFCC θα μπορούσαν να είναι η βάση του classification ειδικά εφόσον μετασχηματιστούν σε κάτι άλλο ή από αυτά εξαχθούν ορισμένα στατιστικά χαρακτηριστικά, πράγμα που κάνουμε παρακάτω με την μέση τιμή και την τυπική απόκλισή τους.
Στη συνέχεια, υπολογίζουμε τα Mel Filterbank Spectral Coefficients (MFSCs) για δύο εκφωνήσεις των ίδιων ψηφίων από δύο διαφορετικούς ομιλητές και υπολογίζουμε τον πίνακα συνδιακύμανσης τους για κάθε εκφώνηση με την συνάρτηση corrcoef της numpy, παρουσιάζοντας το τελικό αποτέλεσμα σε heatmap της βιβλιοθήκης seaborn.



Ύστερα, κάνουμε το ίδιο για τα MFCCs.



Παρατηρούμε ότι τα MFCCs έχουν πολύ μικρότερη συσχέτιση μεταξύ τους. Για αυτόν τον λόγο, όταν κάνουμε επιλογή χαρακτηριστικών, χρησιμοποιούμε τα MFCCs αντί τα MFSCs, αφού στους περισσότερους αλγορίθμους προτιμάμε τα χαρακτηριστικά μας να είναι κατά το δυνατόν ανεξάρτητα, και το να είναι ασυσχέτιστα είναι προϋπόθεση για αυτό το γεγονός.

Σημειώνουμε ότι τα MSFCs τα λάβαμε με την εντολή `librosa.feature.melspectrogram(y=wav, sr=Fs, win_length=win_len, hop_length=step, n_mels=n_features)`. Ισοδύναμα, θα μπορούσαμε να τα είχαμε λάβει με την εντολή `librosa.feature.inverse.mfcc_to_mel(mfcc)`, αφού η βιβλιοθήκη librosa έχει υλοποιημένη την μετατροπή MFCCs σε MFSCs, ή με την εντολή `librosa.db_to_power(scipy.fft.idct(mfcc, type=2, norm="ortho", axis=0))`, αφού τα MFSCs προκύπτουν με τον ίδιο ακριβώς τρόπο με τα MFCCs αλλά χωρίς την εφαρμογή του τελευταίου DFT, οπότε με Inverse DFS μπορούμε να τον αναιρέσουμε και να πάρουμε MFSCs από MFCCs. Την τριπλή αυτή ισοδυναμία επιβεβαιώσαμε και πειραματικά. Προτιμήσαμε τον πρώτο τρόπο, με το melspectrogram εφαρμοσμένο στο wav, γιατί εφαρμόζει τον ευθύ μετασχηματισμό, οπότε δεν υποφέρει από τα επιπλέον αριθμητικά σφάλματα που εισάγουν οι DCT και Inverse DCT.

Βήμα 5

Αρχικά χρησιμοποιούμε NumPy για να ενώσουμε όλα τα χαρακτηριστικά του input που έχουμε υπολογίσει (mfcc, delta, delta-deltas) και να φτιάξουμε έναν πίνακα. Κατόπιν υπολογίζουμε για καθένα από αυτά τη μέση τιμή και την τυπική απόκλιση και επιλέγουμε αυτά ως τα τελικά χαρακτηριστικά που θα χρησιμοποιήσουμε για το classification. Στα σχόλια έχουμε το shape των tensors που έχουμε εκτυπώσει για να είμαστε σίγουροι για την ορθότητα των βημάτων μας.

```
def combine_features(mfccs, deltas, delta_deltas, digits):
    statistical_characteristics = []

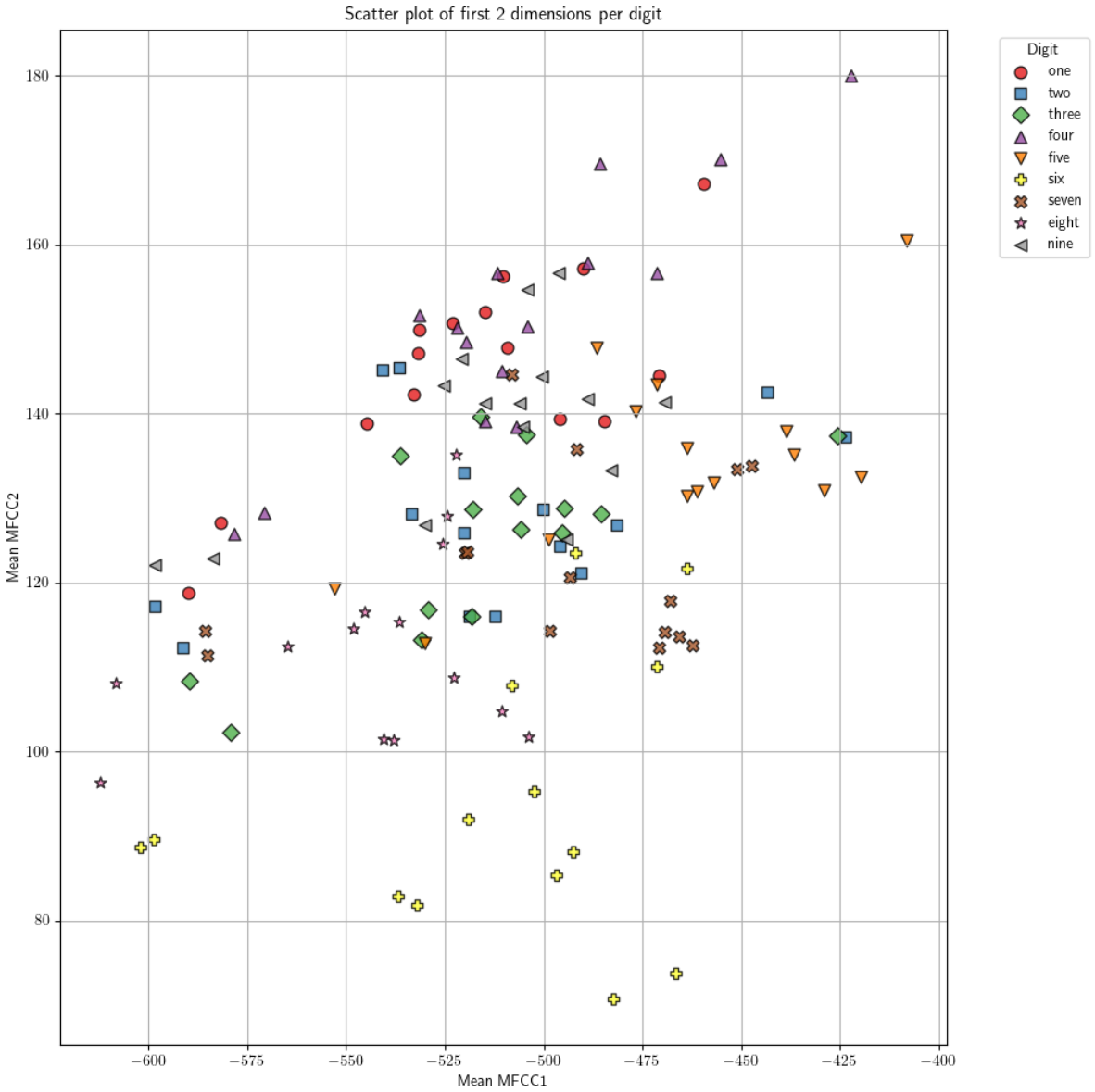
    for mfcc, delta, delta_delta in zip(mfccs, deltas, delta_deltas):
        features = np.vstack((mfcc, delta, delta_delta))   # shape -> (39, smth
    )

        mean = np.mean(features, axis=1) # axis=1 -> (39,)
        std = np.std(features, axis=1)

        characteristics = np.concatenate((mean, std)) # (78,)
        statistical_characteristics.append(characteristics)

    feature_vectors = np.array(statistical_characteristics)  # (133, 78)
```

Οι 2 πρώτες διαστάσεις που καλούμαστε να παραστήσουμε σε scatter plot είναι οι μέσες τιμές των πρώτων 2 MFCC. Το αποτέλεσμα φαίνεται παρακάτω (ο λόγος που χρησιμοποιούμε διαφορετικά χρώματα και σχήματα είναι για να κάνουμε τις κλάσεις μας ευδιάκριτες):

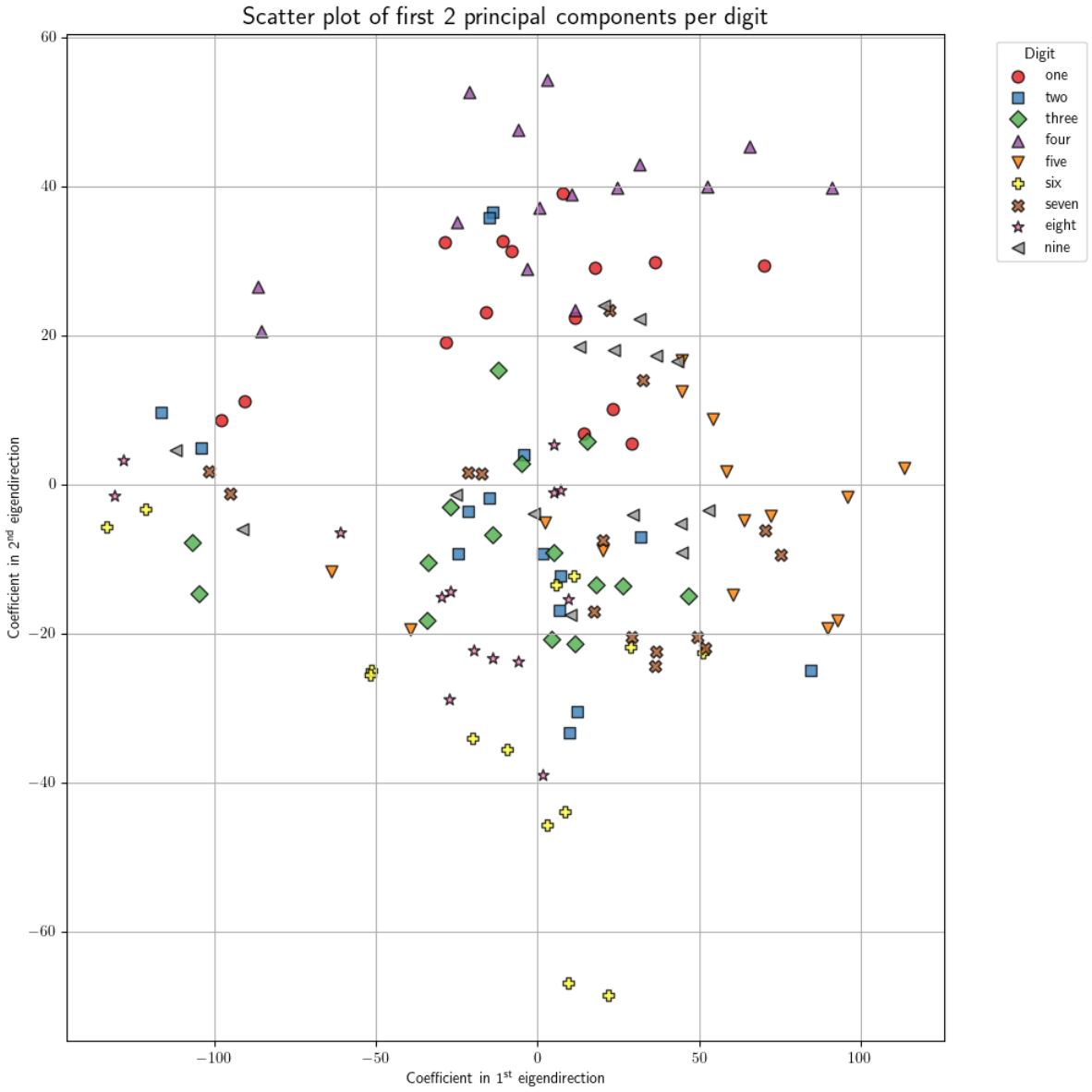


Ουσιαστικά, αυτό που κάνουμε είναι να βάζουμε 2 "τυχαία" (έστω τα 2 πρώτα) features στο scatter plot για να δούμε ότι οι κλάσεις μας δεν είναι τόσο διαχωρίσιμες. Προετοιμαζόμαστε να συγκρίνουμε το αποτέλεσμα με το PCA στο βήμα 6 όπου τα 2 πρώτα features γίνονται εκείνα που εξηγούν το μεγαλύτερο ποσοστό του variance και έτσι κάνουν τις κλάσεις μας πιο διαχωρίσιμες από αυτό που βλέπουμε παραπάνω.

Βήμα 6

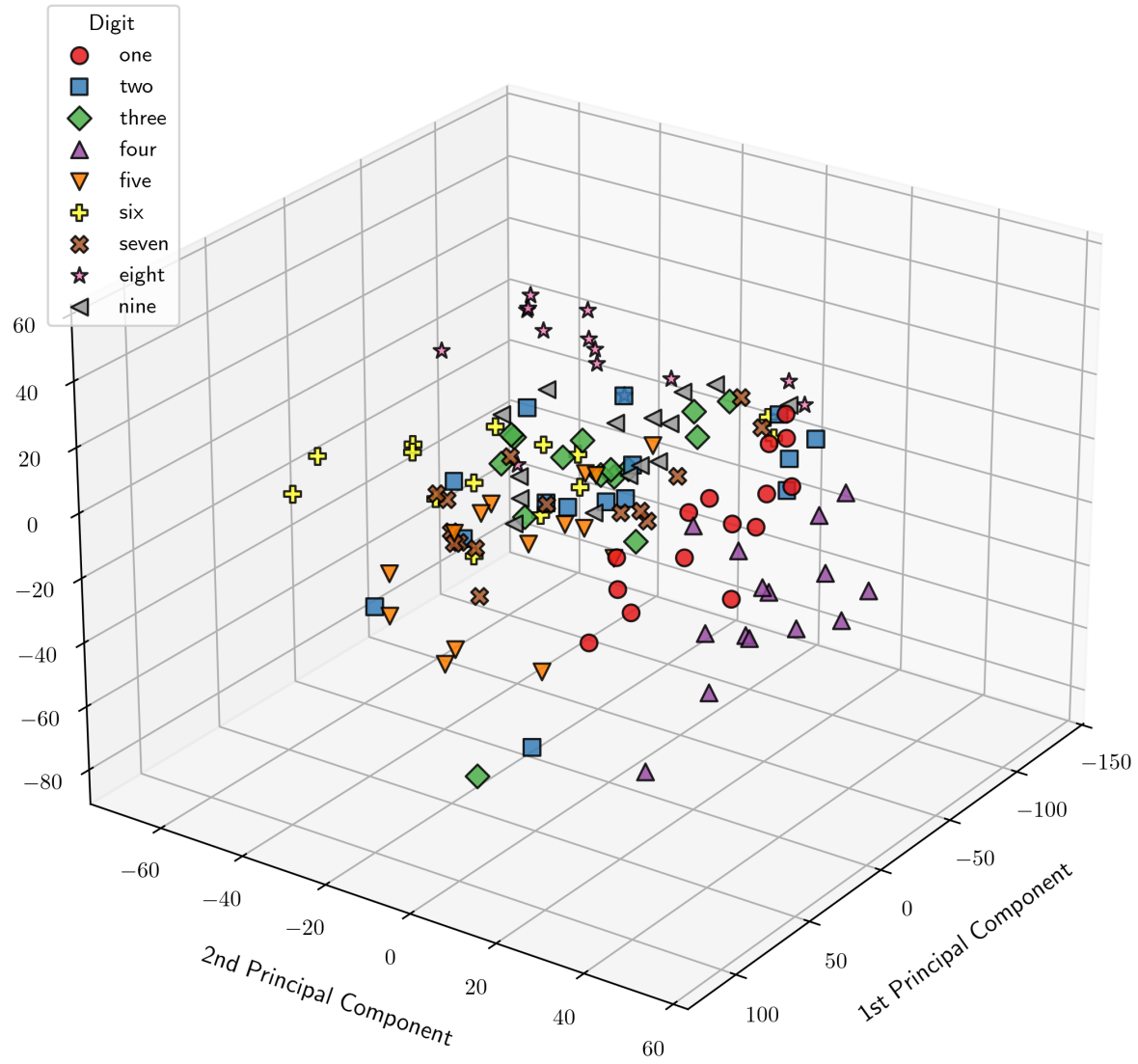
Στο βήμα αυτό, εφαρμόζουμε Principal Component Analysis (PCA) για την μείωση διαστατικότητας των δεδομένων. Αρχικά, μειώνουμε τις διαστάεις των διανυσμάτων χαρακτηριστικών σε 2 με την εντολή `PCA(n_components=2).fit_transform(feature_vectors)` και στη συνέχεια σε 3 θέτοντας n_components = 3.

Παρακάτω παραθέτουμε εκ νέου το scatter plot με τις δύο διαστάσεις των μετασχηματισμένων δεδομένων για n_components = 2.



Παρατηρούμε ότι τα σημεία ακόμα δεν είναι ιδιαίτερα διαχωρισμένα, αλλά πολύ περισσότερο από ό,τι στο Βήμα 5 χωρίς PCA. Τα ψηφία είναι καλύτερα μαζεμένα σε περιοχές, αλλά υπάρχουν πολλές επικαλύψεις ακόμα. Το ποσοστό της συνολικής ερμηνευόμενης διασποράς των δεδομένων είναι 70.5% (άθροισμα στοιχείων πεδίου explained_variance_ratio_).

Ακολούθως, παραθέτουμε το scatter plot με τις τρεις διαστάσεις των μετασχηματισμένων δεδομένων για n_components = 3.



Πλέον, παρατηρούμε ότι τα ψηφία είναι πολύ καλύτερα διαχωρισμένα. Είναι μαζεμένα σε συμπαγείς περιοχές και υπάρχουν πολύ λιγότερες επικαλύψεις, αν και ακόμα υπάρχουν. Επιπλέον, η συνολική ερμηνευόμενη διασπορά είναι 81.4%. Ακόμα, το πεδίο explained_variance_ratio_ μας δίνει το ποσοστό της ερμηνευόμενης διασποράς από κάθε συνιστώσα του PCA, δηλαδή τη διασπορά σε στον κάθε άξονα των πρώτων ιδιοδιανυσμάτων του πίνακα συνδιασποράς. Αυτή είναι πολύ μεγάλη για το πρώτο διάνυσμα και μονότονα μειώνεται όσο αυξάνονται οι συνιστώσες.

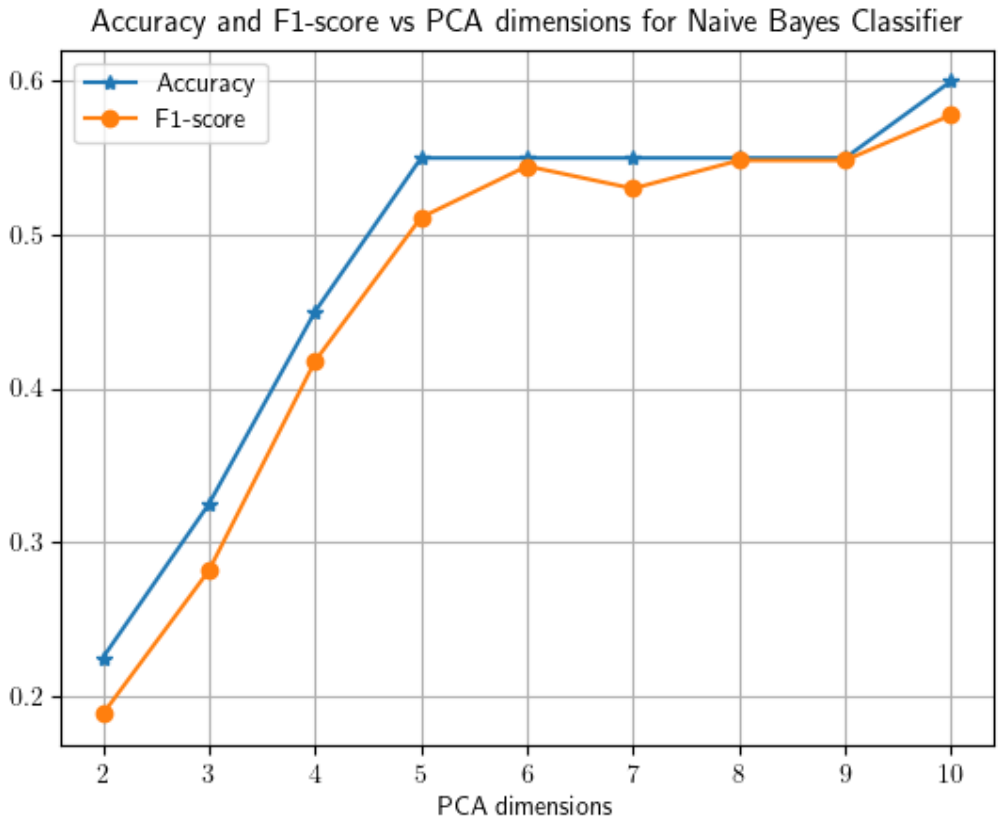
Μπορούμε να θεωρήσουμε ότι η μείωση διαστατικότητας ήταν επιτυχημένη, αφού ερμηνεύσαμε το 70.5% και 81.4% με μόνο 2 και 3 συνιστώσες αντίστοιχα, αλλά για να έχουμε ακόμα καλύτερη απόδοση, θα βοηθούσε να έχουμε λίγες ακόμα συνιστώσες ώστε να ερμηνεύσουμε ακόμα μεγαλύτερο τμήμα της διασποράς και να διαχωριστούν πλήρως τα δεδομένα.

Βήμα 7

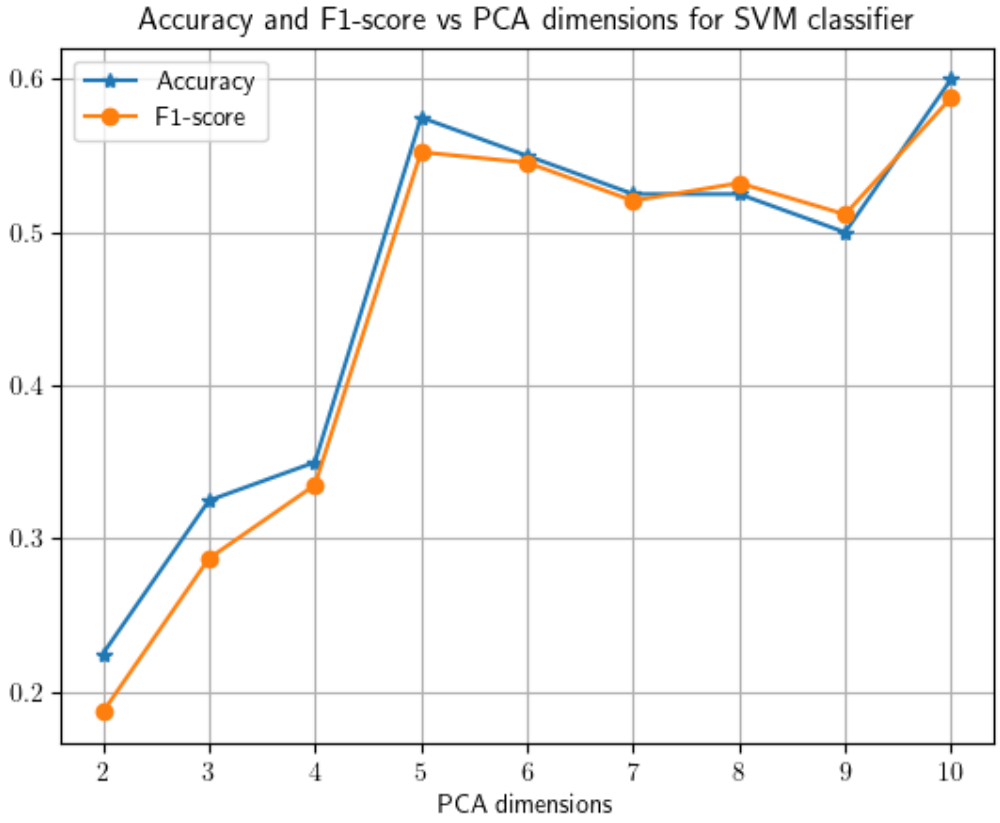
Στο βήμα αυτό, αρχικά χωρίζουμε τα δεδομένα μας σε δεδομένα εκπαίδευσης (train, 70%) και δεδομένα ελέγχου (test, 30%) με χρήση της συνάρτησης train_test_split. Προτού δοκιμάσουμε οποιοδήποτε μοντέλο κανονικοποιούμε τα δεδομένα για να τα χειρίζονται "δίκαια" μέθοδοι επιλογής χαρακτηριστικών όπως η PCA (που χρησιμοποιούμε ξανά) και μοντέλα ταξινόμησης που βασίζονται σε μετρικούς χώρους.

Ακολούθως, σε όλα τα μοντέλα μας θα δοκιμάζουμε ως υπερπαράμετρο το πλήθος συνιστωσών του PCA, από 2 έως και 10. Όλα τα μοντέλα θα αξιολογούνται στο τέλος με βάση το accuracy και το F1-score. Το F1-score το χρησιμοποιούμε κυρίως σε περιπτώσεις που υπάρχουν ανισορροπίες στην εκπροσώπηση των κλάσεων στα δεδομένα, κάτι που παρατηρούμε ότι εδώ δεν συμβαίνει. Για αυτό, σε όλα τα μοντέλα που εξετάζουμε, το accuracy και το f1-score είναι πολύ κοντά και έχουν την ίδια τάση.

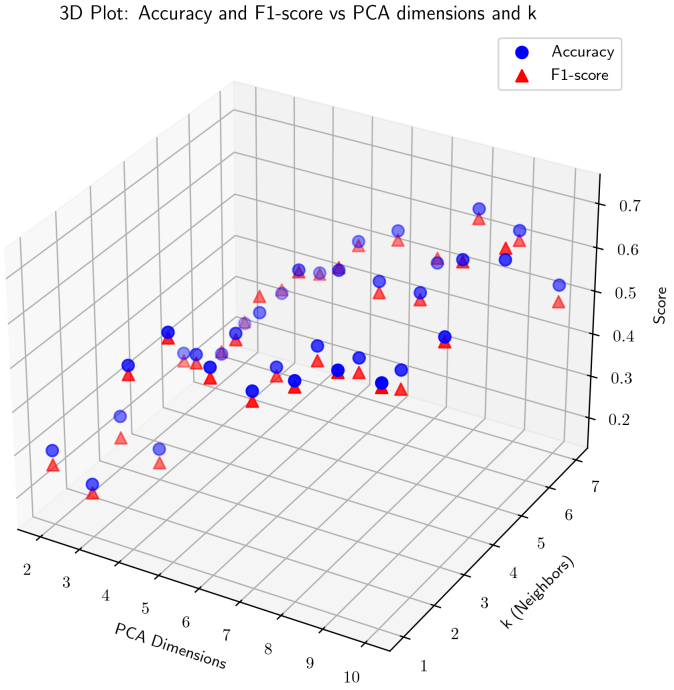
Το πρώτο μοντέλο που δοκιμάζουμε είναι ο Naive Bayes για Γκαουσιανές τυχαίες μεταβλητές (`GaussianNB()`). Παραθέτουμε το διάγραμμα των μετρικών ως προς το πλήθος συνιστωσών κατά το PCA. Το καλύτερο μοντέλο λήφθηκε για 10 συνιστώσες με ακρίβεια 60%.



Το δεύτερο μοντέλο που δοκιμάζουμε είναι το Support Vector Machine με το μη γραμμικό απειροδιάστατο RBF kernel (`SVC()`). Παραθέτουμε και εδώ το αντίστοιχο διάγραμμα. Το καλύτερο μοντέλο λήγθηκε πάλι για 10 συνιστώσες με ακρίβεια 60%.

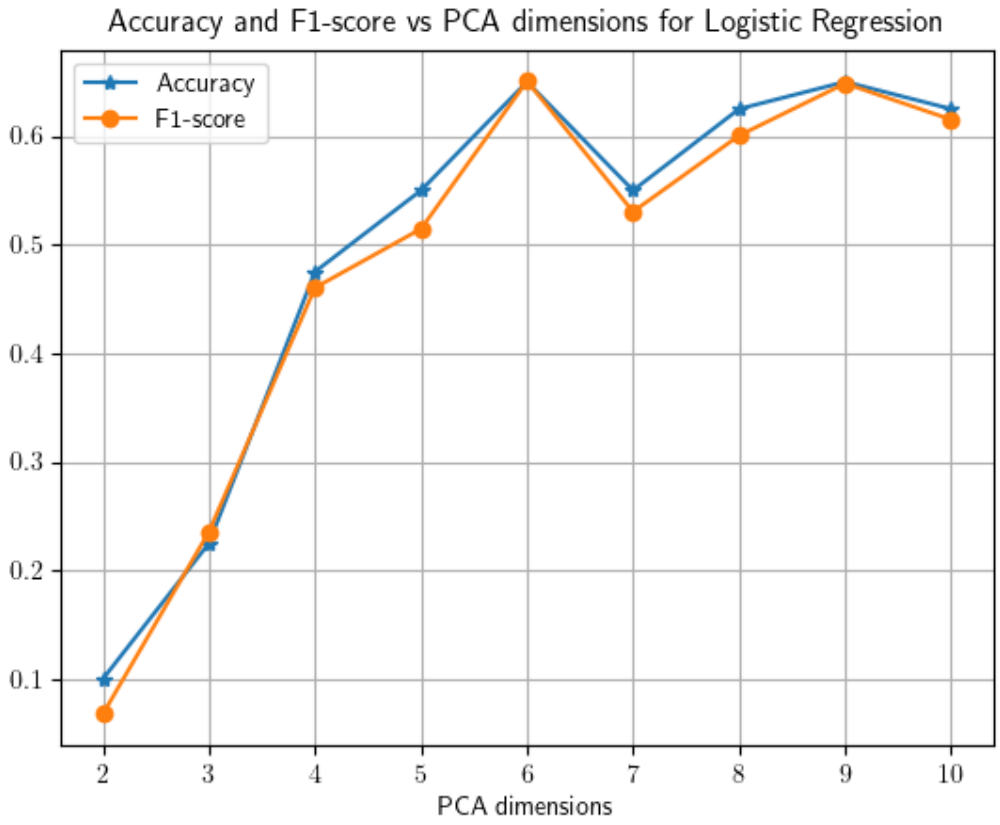


Το τρίτο μοντέλο που δοκιμάζουμε είναι το k-Nearest Neigbors, με επιπλέον υπερπαράμετρο το $k \in \{1, 3, 5, 7\}$ (`KNeighborsClassifier(n_neighbors=k)`). Παραθέτουμε το αντίστοιχο τριδιάστατο διάγραμμα. Το καλύτερο μοντέλο λήφθηκε για 10 συνιστώσες και k = 5 με ακρίβεια 70%.



Το τελευταίο μοντέλο που δοκιμάζουμε είναι το Logistic Regression (`LogisticRegression()`). Παρα-

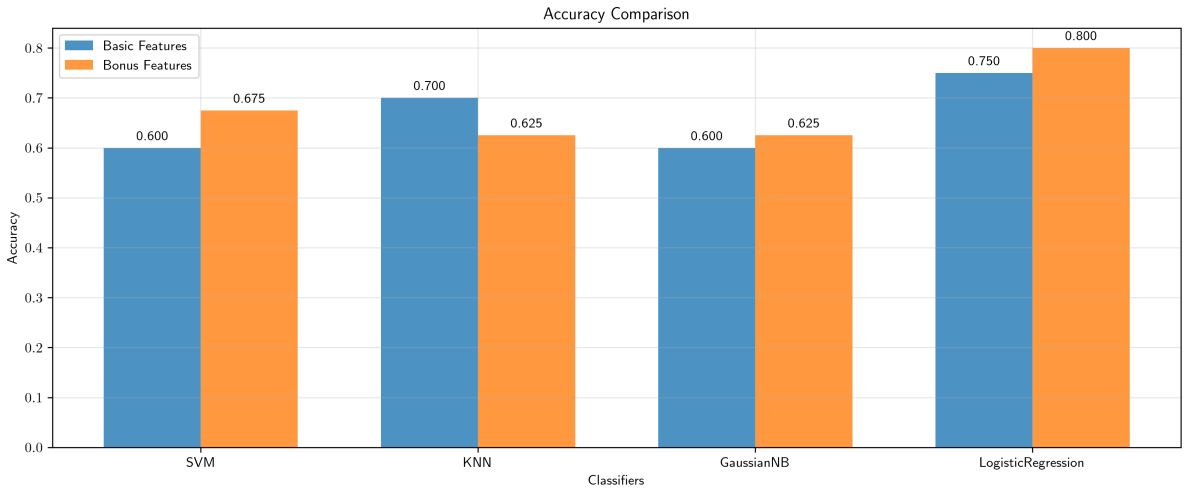
θέτουμε πάλι το αντίστοιχο διάγραμμα. Το καλύτερο μοντέλο λήφθηκε για 6 συνιστώσες με ακρίβεια 65%.

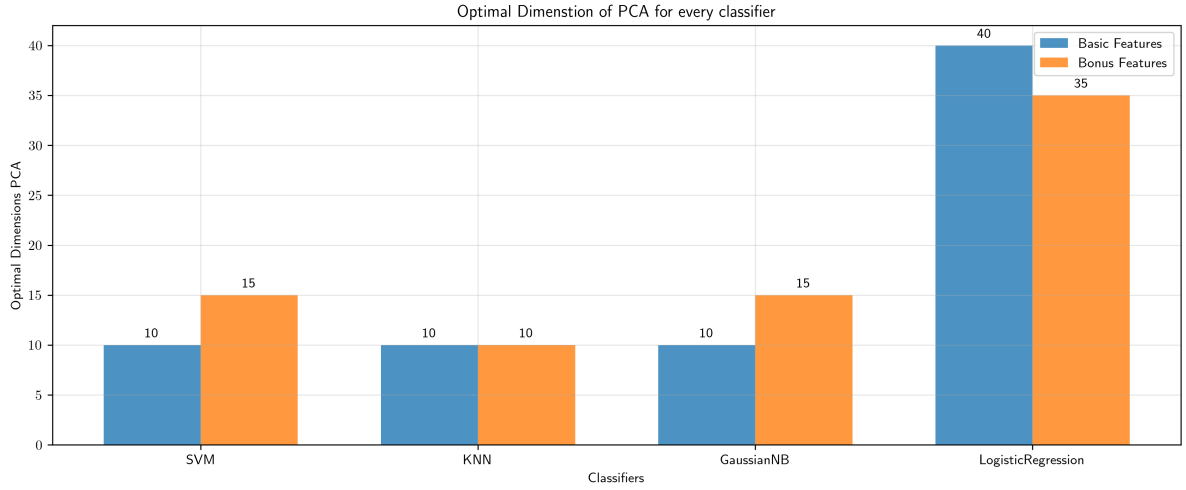


Bonus

Αυτό που θα κάνουμε είναι να προσθέσουμε επιπλέον πληροφορία στο διάνυσμα χαρακτηριστικών (εκτός από mean mfcc, delta και delta_deltas) όπως το zero-crossing rate. Για αυτό τροποποιούμε την συνάρτηση εξαγωγής των mfcc και των παραγώγων τους έτσι ώστε εκτός από αυτά να μας επιστρέφει και τα zero crossings και spectral centroids. Τέλος υλοποιούμε και μία συνάρτηση για την σύγκριση της επίδοσης των ταξινομητών που χρησιμοποιήσαμε παραπάνω σε αυτά τα νέα επαυξημένα δείγματα. Τα βήματα που ακολουθούμε scaling και PCA είναι ίδια με πριν. Γενικά, αυτό που ελπίζουμε εισάγοντας επιπλέον χαρακτηριστικά είναι να κάνουμε τα δείγματά μας περισσότερο διαχωρίσιμα. Αυτό πράγματι θα γίνει εφόσον τα χαρακτηριστικά αυτά που θα επιλέξουμε είναι ασυσχέτιστα μεταξύ τους (δεν κουβαλούν redundant πληροφορία), αλλά και ασυσχέτιστα με εκείνα που έχουμε ήδη επιλέξει. Προφανώς, όποια χαρακτηριστικά και να επιλέξουμε να προσθέσουμε δεν θα πετύχουμε και τα 2 στον μέγιστο βαθμό. Έτσι, αναμένουμε οι μετρικές μας να βελτιωθούν αλλά όχι δραματικά. Στις δοκιμές που κάναμε το καλύτερο αποτέλεσμα πάρθηκε από τον ταξινομητή Logistic Regression - Accuracy 80%. Επιπλέον, η προσδοκία μας, κατά κανόνα, επιβεβαιώνεται αφού η ακρίβεια βελτιώνεται, αλλά όχι πάνω από 7.5%. Στη επόμενη σελίδα παρουσιάζουμε μία σύγκριση της απόδοσης των ταξινομητών πριν και μετά την εισαγωγή των επιπλέον features αλλά και το πόσες διαστάσεις είναι καλύτερο να διατηρήσουμε μετά το PCA.

Αξίζει να επισημάνουμε ότι το Logistic Regression παρουσιάζει και χωρίς τα επιπλέον features καλύτερη επίδοση και από το μη bonus ερώτημα. Αυτό γίνεται διότι ο αριθμός των διαστάσεων χαρακτηριστικών που του δίνονται είναι 40 και 35 αντίστοιχα, πολύ μεγαλύτερος από το προηγούμενο μέγιστο, δηλαδή 10.



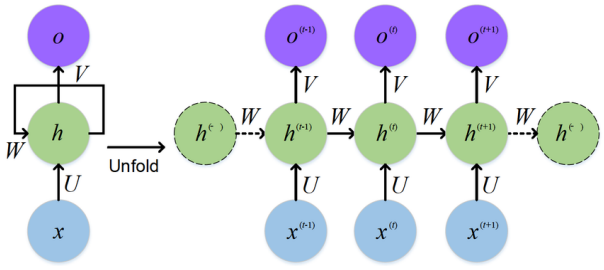


Βήμα 8

Σε αυτό το βήμα μας ζητείται να δημιουργήσουμε ακολουθίες 10 σημείων ενός ημιτόνου και ενός συνημιτόνου συχνότητας 40Hz. Στόχος είναι να προβλέψουμε τις ακολουθίες συνημιτόνου έχοντας ως δεδομένες εκείνες των ημιτόνων.

Για να το πετύχουμε αυτό επιστρατεύουμε 3 είδη νευρωνικών δικτύων, το RNN, το LSTM και το GRU. Για να τα ορίσουμε και να τα εκπαιδεύσουμε αξιοποιούμε την PyTorch.

- **RNN:** Πρόκειται για ένα τύπο νευρωνικού δικτύου ειδικά σχεδιασμένου ώστε να επεξεργαστούμε ακολουθιακά δεδομένα και να ανιχνεύσουμε τις εξαρτήσεις τους και τα μοτίβα τους στο χρόνο.



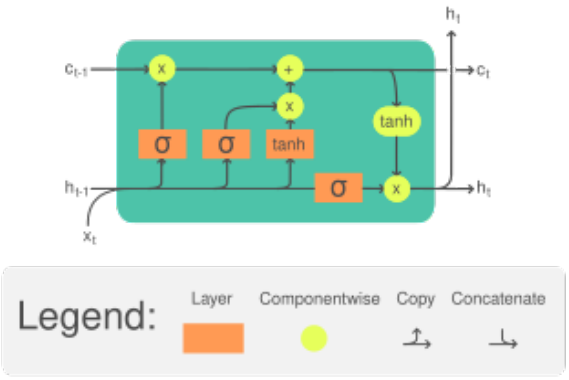
Ο κώδικας του ορισμού του είναι ο εξής:

```python
class myRNN(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, output_size=1):
        super(myRNN, self).__init__()
        self.rnn = nn.RNN(input_size=input_size, hidden_size=hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.rnn(x)
        out = self.fc(out)
        return out
```

Ουσιαστικά έχουμε το component που υλοποιοεί ένα RNN, το οποίο πρσφέρεται έτοιμο από την PyTorch και κατόπιν ένα fully connected layer το οποίο συνδυάζει τα outputs των hidden layers του RNN κάθε χρονική στιγμή-δείγμα και μας δίνει το τελικό προβλεπόμενο output για τη δεδομένη χρονική στιγμή.

- **LSTM:** Το LSTM είναι ένα είδος RNN το οποίο αναπτύχθηκε προκειμένου να αντιμετωπίσει το πρόβλημα του vanishing gradient που εμφανιζόταν έντονα στον πρόγονό του. Ουσιαστικά, δίνει δυνατότητα short term memory στο μοντέλο ώστε να "θυμάται" πληροφορίες και εξαρτήσεις στα δεδομένα για χιλιάδες δείγματα, σε αντίθεση με το RNN το οποίο "έχανε" τις εξαρτήσεις δεδομένων που απείχαν κάποιο μεγάλο αριθμό δειγμάτων (αυτό είναι το manifestation του vanishing gradient).

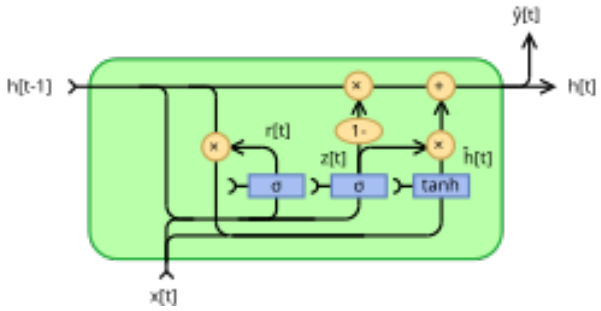


Ο κώδικας του ορισμού του φαίνεται παρακάτω και ακολουθεί τη λογική του RNN:

```python
class myLSTM(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, num_layers=1, output_size=1): # hidden_size = 32 for moderate expressiveness
        super(myLSTM, self).__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True) # input is in format (batch_size, seq_len, input_size)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.fc(out)
        return out
```

- **GRU:** Είναι και αυτό "απόγονος" του RNN. Πρακτικά μοιάζει με το LSTM, μόνο που έχει έναν μηχανισμό από πύλες στο input, αλλά δεν έχει πύλη στο output και συνεπώς έχει λιγότερες παραμέτρους από το LSTM.



Ο κώδικας του ορισμού του φαίνεται παρακάτω και ακολουθεί τη λογική του RNN και του LSTM:

```
class myGRU(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, output_size=1):
        super(myGRU, self).__init__()
        self.gru = nn.GRU(input_size=input_size, hidden_size=hidden_size,
    batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.gru(x)
        out = self.fc(out)
        return out
```

Στην συνέχεια, εκπαιδεύουμε τα μοντέλα μας και συγκρίνουμε την απόδοσή τους. Η διαδικασία που ακολουθούμε είναι η εξής:

1. **Δημιουργία δειγμάτων ημιτόνων και συνημιτόνων:**
   Με την NumPy δημιουργούμε τα σήματα-εισόδους για τα νευρωνικά δίκτυα μας καθώς και τα σήματα-στόχους που θέλουμε να προβέψουμε (t = np.linspace(0, 1, total_points), x = np.sin(2*np.pi*f*t), y = np.cos(2*np.pi*f*t))

2. **Εκπαίδευση των μοντέλων:**

```
def train_model(model, X_train, Y_train, epochs=100, lr=0.001):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr) # default
    learning rate small to avoid overshooting
    losses = []

    for epoch in range(epochs):
        model.train() # put model in trainning mode
        optimizer.zero_grad()   # clean accumulated gradients

        outputs = model(X_train)
        loss = criterion(outputs, Y_train)

        loss.backward()   # backpropagate loss gradients
        optimizer.step()  # update weights

        losses.append(loss.item())

        if (epoch + 1) % 20 == 0:
            print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.6f}')

    return losses
```

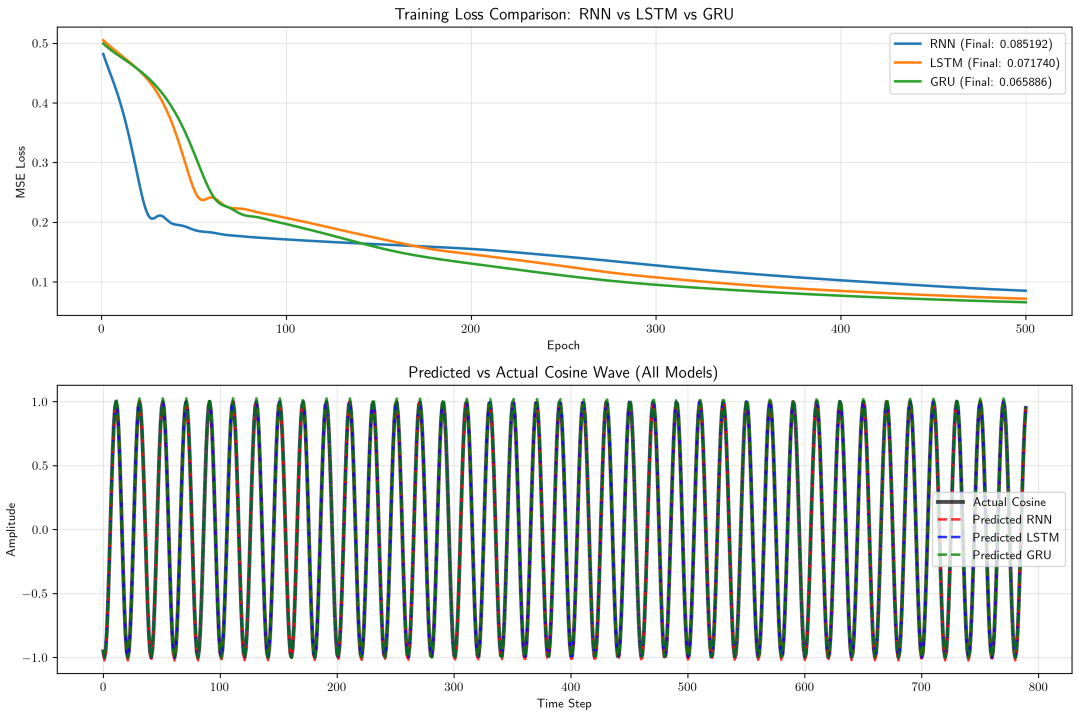
Ως κριτήριο ελαχιστοποίησης επιλέξαμε το Mean Square Error ενώ για optimizer τον Adam. Ο Adam ενσωματώνει και όρους ορμής στην δική του υλοποίηση του gradient descend. Εκπαιδεύουμε όλα τα μοντέλα για 100 εποχές.
Αρχικά, πριν από κάθε εποχή πρέπει να καθαρίσουμε τα προηγούμενα gradients γιατί η default συμπεριφορά της PyTorch είναι να τα συσσωρεύει, όχι να τα πανωγράφει.
Στη συνέχεια, υπολογίζουμε το output του μοντέλου και το loss (αντίστοιχο της απόκλισης από το στόχο μας). Τέλος κάνουμε backpropagation τα loss gradients και ενημερώνουμε τα βάρη.
Τα losses αποθηκεύονται ώστε να χρησιμοποιηθούν σε διαγράμματα μετέπειτα.

3. **Αξιολόγηση των μοντέλων:**

```
def evaluate_model(model, X_test, Y_test):
    model.eval()    # put model in eval mode
    with torch.no_grad():   # we do not need to calculate and store
    gradients in inference
        predictions = model(X_test)
        mse = nn.MSELoss()(predictions, Y_test)
        mae = torch.mean(torch.abs(predictions - Y_test))
    return predictions, mse.item(), mae.item()
```

Αρχικά, βάζουμε το μοντέλο σε eval mode (από documentation της PyTorch) και κατόπιν απενεργοποιούμε τον υπολογισμό και την αποθήκευση των gradients διότι αυτή η υπολογιστικά ακριβή διαδικασία δεν είναι απαραίτητη κατά το inference. Τέλος υπολογίζουμε τις μετρικές μας για τα test samples και τις επιστρέφουμε.

Αποτελέσματα



Σε αυτό το γράφημα εξετάζουμε τη συμπεριφορά των μοντέλων κατά την εκπαίδευσή τους και παίρνουμε μια πρώτη εικόνα για το κατά πόσο το αποτέλεσμα είναι ικανοποιητικό.

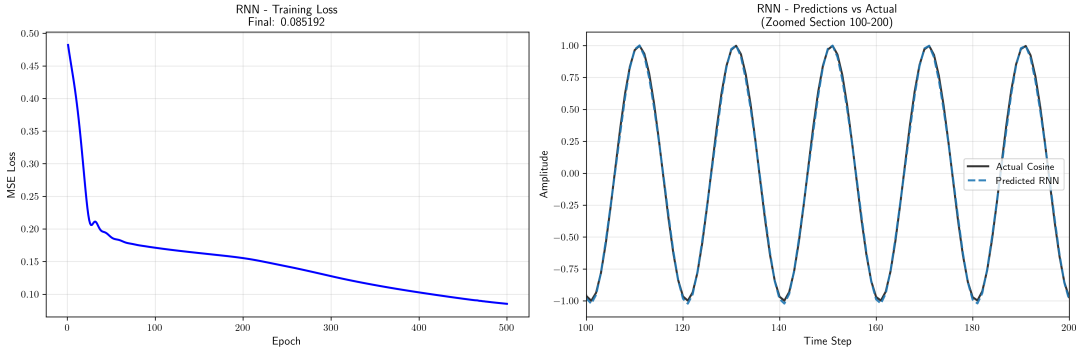
Βλέπουμε ότι έχουμε εκπαιδεύσει τα μοντέλα μας για 500 εποχές. Καταλήξαμε σε αυτή την επιλογή ως εξής:

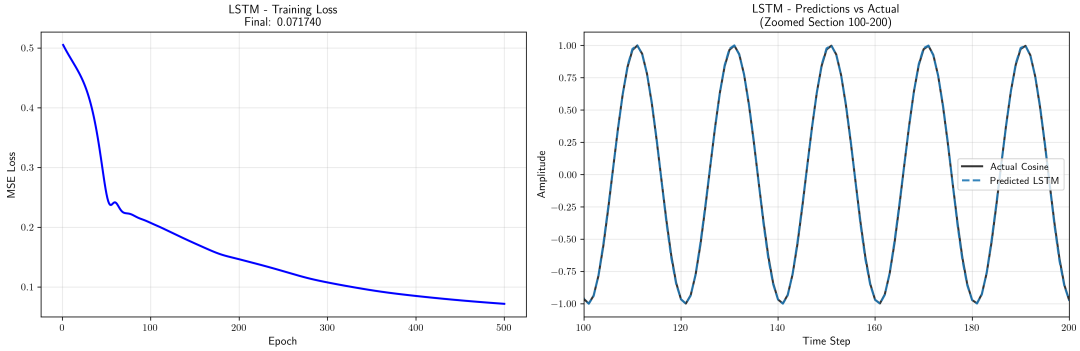
1. Αρχικά, εκπαιδεύσαμε με 100 εποχές. Αυτό οδήγησε στο εξής παράδοξο: Το RNN, θεωρητικά το πιο "αδύναμο" μοντέλο είχε το μικρότερο loss - δηλαδή είχε προσαρμοστεί καλύτερα στα δεδομένα. Αυτό μπορεί να έχει μόνο μία εξήγηση, δεδομένου ότι έχει μικρότερη χωρητικότητα από τα προηγούμενα 2. Ο μεγαλύτερος αριθμός παραμέτρων των LSTM και GRU δεν τους έχει επιτρέψει να συγκλίνουν ακόμα. Οπότε αυξάνουμε τον αριθμό εποχών σε 300.

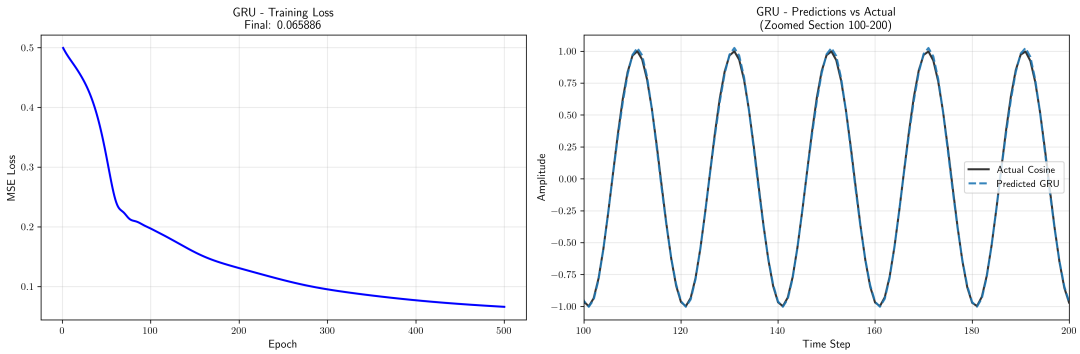
2. Όπως φαίνεται και στο plot παραπάνω, μετά τις 180 εποχές πράγματι τόσο το LSTM όσο και το GRU έχουν μικρότερο loss από το RNN. Αλλά βλέπουμε ότι το loss έχει την τάση να μειωθεί κι άλλο. Οπότε αυξάνουμε σε 500 εποχές

3. Βλέπουμε ότι το loss δεν βελτιώνεται οπότε σταματάμε (σε αυτό το σημείο θα μπορούσαμε να αρχίσουμε να αυξάνουμε τα hidden layers).

Αφότου εξηγήσαμε την διαδικασία της εκπαίδευσής μας, βλέπουμε ότι και το αποτέλεσμα για τα μοντέλα μας, οπτικά, είναι αρκετά υποσχόμενο. Για να το επιβεβαιώσουμε πλήρως εξετάζουμε το αποτέλεσμα σε ένα zoom της κυματομορφής για κάθε μοντέλο ξεχωριστά.







Και για τα 3 μοντέλα το αποτέλεσμα είναι πολύ ικανοποιητικό.

Τέλος, επιλέγουμε το καλύτερο μοντέλο με βάση το μικρότερο MSE στο test set. Αυτό βλέπουμε ότι είναι το GRU.

```
============================================================
BEST MODEL SELECTION
============================================================
1. GRU: MSE=0.0651, MAE=0.1122
2. LSTM: MSE=0.0712, MAE=0.1169
3. RNN: MSE=0.0847, MAE=0.1426

Best Model: GRU
============================================================
```

Βήμα 9

Πρώτα από όλα κάνουμε clone τα δεδομένα σε μορφή .wav από το repository της εργασίας στον υπολογιστή μας.

Στη συνέχεια, αξιοποιούμε τον βοηθητικό κώδικα του εργαστηρίου προκειμένου να υλοποιήσουμε τα επόμενα βήματά μας (*parser.py, hmm.py*)

```python
def create_data():
    X, X_test, y, y_test, spk, spk_test = parser("recordings", n_mfcc=13)

    # TODO: YOUR CODE HERE
    (
        X_train,
        X_val,
        y_train,
        y_val,
        spk_train,
        spk_val,
    ) =  train_test_split(X, y, spk, test_size=0.2, random_state=42, stratify=y
    ) # split X into a 80/20 train validation split

    # Scaling data based on the training set
    scale_fn = make_scale_fn(X_train)
    X_train = scale_fn(X_train)
    X_val = scale_fn(X_val)
    X_test = scale_fn(X_test)

    train_dic = gather_in_dic(X_train, y_train, spk_train)
    val_dic = gather_in_dic(X_val, y_val, spk_val)
    test_dic = gather_in_dic(X_test, y_test, spk_test)
    labels = list(set(y_train))

    return train_dic, y_train, val_dic, y_val, test_dic, y_test, labels
```

Όπως φαίνεται και στη συνάρτηση παραπάνω, διαβάζουμε από το φάκελο τα wav αρχεία και εξάγουμε 13 MFCC για το καθένα. Στη συνέχεια με την έτοιμη συνάρτηση split του κώδικα τη χωρίζουμε στο default train test split το οποίο έχει 2700 από τα 3000 δείγματα ακολουθιών στο train και τα υπόλοιπα στο test. Στη συνέχεια, καλούμαστε να χωρίσουμε το test set αυτό σε 2 μικρότερα. Ένα validation και ένα test set σε αναλογία 80-20 αντίστοιχα. Αυτό, όπως φαίνεται και στον κώδικα παραπάνω, γίνεται με τη χρήση του *train_test_split*. Η παράμετρος stratify μας εξασφαλίζει την ομοιόμορφη κατανομή των δειγμάτων. Δηλαδή, ότι θα έχουμε τον ίδιο αριθμό ψηφίων από κάθε κατηγορία. Έτσι αποφεύγουμε biases.



Figure 1: Ανάγνωση Dataset

Βήμα 10

Στο βήμα αυτό υλοποιούμε αναγνώριση ψηφίων από ομιλία με GMM-HMM. Η λειτουργία του μοντέλου φαίνεται διαγραμματικά παρακάτω:

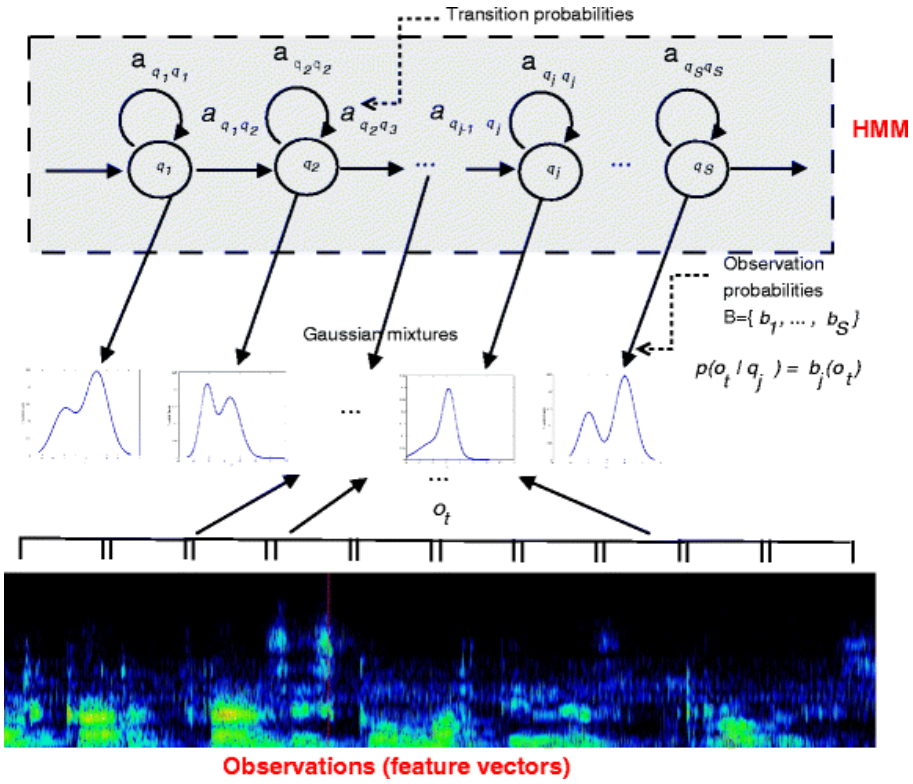


Figure 2: Image by [?]

Δημιουργούμε ένα HMM ανά ψηφίο, οπότε η έξοδος του i-οστού HMM θα είναι η πιθανότητα το σήμα να αντιστοιχεί στο i-οστό ψηφίο. Οι παρατηρήσεις σε κάθε state του HMM είναι ένα διαφορετικό mixture of gaussians. Αφού εκπαιδευτεί το GMM από μόνο του σε όλα τα δεδομένα, θα εκπαιδευτεί από κοινού το GMM-HMM μοντέλο και το μείγμα κατανομών ανά state θα γίνει fit στην παρατήρηση που συμβαίνει σε αυτό το state. Ιδανικά, θα θέλαμε εξ'αρχής να μην εκπαιδεύουμε το GMM σε όλα τα δεδομένα, αλλά μόνο σε αυτά που αντιστοιχούν στο χρονικό διάστημα που περιγράφεται από το state. Κάτι τέτοιο, όμως, είναι δυστυχώς δύσκολο να γίνει, αφού δεν είναι προκαθορισμένο για κάθε ψηφίο το πότε θα γίνει η μετάβαση από το ένα state στο επόμενο, οπότε θα αρκεστούμε στην διαφοροποίηση μεταξύ των GMMs που κάνει στη συνέχεια η από κοινού εκπαίδευση. Τέλος, σημειώνουμε ότι με το μείγμα γκαουσιανών μοντελοποιούμε την κατανομή των MFCCs ανά ψηφίο και state, αφού περιμένουμε πως η κάθε τέτοια κατανομή δεν θα είναι unimodal στον χώρο των MFCCs λόγω των διαφορών στην εκφώνηση του ίδιου ψηφίου μεταξύ διαφορετικών ομιλητών.

Η συνάρτηση train φαίνεται παρακάτω:

```
1 def train_hmms(train_dic, labels, n_states, n_mixtures):
2     hmms = {}  # create one hmm for each digit
3
```

```
    for dig in labels:
        X, _, _, _ = train_dic[dig]
        # TODO: YOUR CODE HERE
        emission_model = None
        if gmm: emission_model = initialize_and_fit_gmm_distributions(X,
    n_states=n_states, n_mixtures=n_mixtures)
        else: emission_model = initialize_and_fit_normal_distributions(X,
    n_states=n_states)
        hmms[dig] = train_single_hmm(X, emission_model, dig, n_states)
    return hmms
```

Την `initialize_and_fit_normal_distributions` καθώς είναι υποπερίπτωση του GMM για n_mixtures = 1.

Η αρχική εκπαίδευση του GMM γίνεται ως εξής:

```
def initialize_and_fit_gmm_distributions(X, n_states, n_mixtures):
    # TODO: YOUR CODE HERE
    dists = []
    all_data = np.concatenate(X, dtype=np.float32)
    D = all_data.shape[1]

    global_mean = np.mean(all_data, axis=0)
    global_std = np.std(all_data, axis=0)

    variance_floor = 1e-6
    base_variance = np.maximum(global_std**2, variance_floor).astype(np.float32
    )

    for _ in range(n_states):
        distributions = []
        for _ in range(n_mixtures):
            means = global_mean + 0.1 * np.random.randn(D).astype(np.float32)

            covs = base_variance.copy()

            distributions.append(
                Normal(means=means, covs=covs, covariance_type=covariance_type)
            )

        gmm_model = GeneralMixtureModel(
            distributions,
            verbose=False,
            max_iter=n_iters,
            random_state=42,
            tol=tol
        ).fit(all_data)
        dists.append(gmm_model)

    return dists
```

Δηλαδή αρχικοποιούμε κάθε γκαουσιανή σε multivariate gaussian με μέσο ειλημμένο από D-διάστατη τυπική κανονική κατανομή, όπου D η διάσταση των δεδομένων, εδώ 13 για τα MFCCs, και με συνδιασπορά τον D-διάστατο ταυτοτικό πίνακα. Επίσης περιορίζουμε κατά την εκπαίδευση τον πίνακα συνδιασποράς να είναι διαγώνιος, ώστε να μειώσουμε τις παραμέτρους και τον χρόνο εκπαίδευσης αλλά και να έχουμε πιο απλές αναπαραστάσεις για τα δεδομένα μας.

Αντίστοιχα, το HMM το εκπαιδεύουμε ως εξής:

```
def train_single_hmm(X, emission_model, digit, n_states):
    A = initialize_transition_matrix(n_states)
    start_probs = initialize_starting_probabilities(n_states)
    end_probs = initialize_end_probabilities(n_states)
```

```
    data = [x.astype(np.float32) for x in X]

    model = DenseHMM(
        distributions=emission_model,
        edges=A,
        starts=start_probs,
        ends=end_probs,
        verbose=True,
        max_iter=n_iters,
        tol=tol,
        random_state=42
    ).fit(data)
    return model
```

Για πίνακα μετάβασης χρησιμοποιήσαμε τον $A = [\alpha_{i,j}]_{1 \le i,j \le n}$, όπου $n = \text{n_states}$ το πλήθος καταστάσεων στα HMMs, με:

$$\alpha_{i,j} = \begin{cases} 1, & \text{if } j = i = n \\ 0.8 & \text{if } j = i \neq n \\ 0.2 & \text{if } j = i + 1 \\ 0 & \text{αλλιώς} \end{cases}$$

Οι τιμές 0.8 για την ανακύκλωση και 0.2 για την μετάβαση επιλέχθηκαν επειδή το μοντέλο πιο συχνά θα επιλέγει να μείνει στην ίδια κατάσταση, παρά να μεταβεί στην επόμενη.

```
def initialize_transition_matrix(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    A = np.full((n_states, n_states), 1e-6 ,dtype=np.float32)
    for i in range(n_states):
        if i+1 < n_states:
            A[i][i] = np.float32(0.8)
            A[i][i+1] = np.float32(0.2)
        else:
            A[i][i] = np.float32(1.0)
    return A
```

Τέλος, αρχικοποιήσαμε τις πιθανότητες εκκίνησης ως 1 στην αρχική κατάσταση και 0 σε όλες τις άλλες, ενώ τις πιθανότητες τερματισμού ως 1 στην τελική κατάσταση και 0 σε όλες τις άλλες.

```
def initialize_starting_probabilities(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    pi = np.zeros(n_states, dtype=np.float32)
    pi[0] = np.float32(1.0)
    return pi


def initialize_end_probabilities(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    pi_end = np.zeros(n_states, dtype=np.float32)
    pi_end[-1] = np.float32(1.0)
    return pi_end
```

Βήμα 11

Πλέον, έχουμε υλοποιήσει τα μοντέλα, οπότε μπορούμε να τα εκπαιδεύσουμε στο training set και να ελέγξουμε την απόδοση του καθενός στο validation set. Η εκπαίδευση κάθε GMM-HMM γίνεται με τον αλγόριθμο Expectation-Maximization (EM) για $\text{n_iters} = 250$ επαναλήψεις ή μέχρι να υπάρξει σύγκλιση, όπου σύγκλιση συμβαίνει όταν η διαφορά του λογαρίθμου της πιθανότητας

γίνεται μικρότερη σε απόλυτη τιμή από την ανεκτικότητα tol = 0.1. Ελέγχουμε την απόδοση του μοντέλου για πλήθος καταστάσεων από 1 έως 4 και πλήθος γκαουσιανών στο μείγμα από 1 έως 5. Ο κώδικας για αυτό φαίνεται παρακάτω:

```
def hmm_tuning(max_n_states, max_n_mixtures):
    # Storage for all model metrics
    metrics = {
        'n_states': [],
        'n_mixtures': [],
        'accuracy': [],
        'precision': [],
        'confusion_matrices': [],
        'predictions': [],
        'true_labels': []
    }

    best_accuracy = 0
    best_metrics = {}
    best_model = None

    for n_states in range(1, max_n_states + 1):
        for n_mixtures in range(1, max_n_mixtures + 1):
            # Set seed for reproducibility
            np.random.seed(42)

            # Train and evaluate model
            hmms = train_hmms(train_dic, labels, n_states, n_mixtures)  # Assumes these params are used
            pred_val, true_val = evaluate(hmms, val_dic, labels)

            # Calculate metrics
            val_accuracy = accuracy_score(true_val, pred_val)
            val_precision = precision_score(true_val, pred_val, average='weighted', zero_division=0)
            cm = confusion_matrix(true_val, pred_val)

            # Store metrics
            metrics['n_states'].append(n_states)
            metrics['n_mixtures'].append(n_mixtures)
            metrics['accuracy'].append(val_accuracy)
            metrics['precision'].append(val_precision)
            metrics['confusion_matrices'].append(cm)
            metrics['predictions'].append(pred_val)
            metrics['true_labels'].append(true_val)

            # Update best model
            if val_accuracy > best_accuracy:
                best_accuracy = val_accuracy
                best_metrics = {
                    'n_states': n_states,
                    'n_mixtures': n_mixtures,
                    'accuracy': val_accuracy,
                    'precision': val_precision,
                    'confusion_matrix': cm,
                    'predictions': pred_val,
                    'true_labels': true_val,
                    'model': hmms
                }
                best_model = hmms

    print(f"Best: states = {best_metrics['n_states']}, mixtures = {best_metrics['n_mixtures']}, "
          f"accuracy = {best_metrics['accuracy']:.4f}")
```

```
57
58      return best_metrics, metrics, best_model
```

Βήμα 12

Αφού εκπαιδεύσουμε τα μοντέλα μας, τα κάνουμε evaluate με την ακόλουθη συνάρτηση:

```
1  def evaluate(hmms, dic, labels):
2      pred, true = [], []
3      for dig in labels:
4          X, _, _, _ = dic[dig]
5          for sample in X:
6              ev = [0] * len(labels)
7              sample = np.expand_dims(sample, 0)
8              for digit, hmm in hmms.items():
9                  # TODO: YOUR CODE HERE
10                 logp = hmm.log_probability(sample)  # use the hmm.
       log_probability function
11                 ev[digit] = logp
12
13             # TODO: YOUR CODE HERE
14             predicted_digit = np.argmax(ev)  # Calculate the most probable
       digit
15             pred.append(predicted_digit)
16             true.append(dig)
17     return pred, true
```

Το evaluation γίνεται παίρνοντας τα δεδομένα στο validation set, βάζοντας το καθένα ως είσοδο στο μοντέλο κάθε ψηφίου, λαμβάνοντας από το κάθε μοντέλο τον λογάριθμο τον εκτιμώμενο λογάριθμο της πιθανότητας η είσοδος να αντιστοιχεί σε αυτό το ψηφίο, τυπώνοντας ως έξοδο το ψηφίο που έλαβε την μέγιστη πιθανότητα (λογάριθμο) από το μοντέλο του, και συγκρίνοντας με το πραγματικό ψηφίο αυτής της εισόδου.

Στη συνέχεια, δημιουργούμε διαγράμματα γραμμής και heatmap με το accuracy κάθε διαμόρφωσης μοντέλου στο validation set.

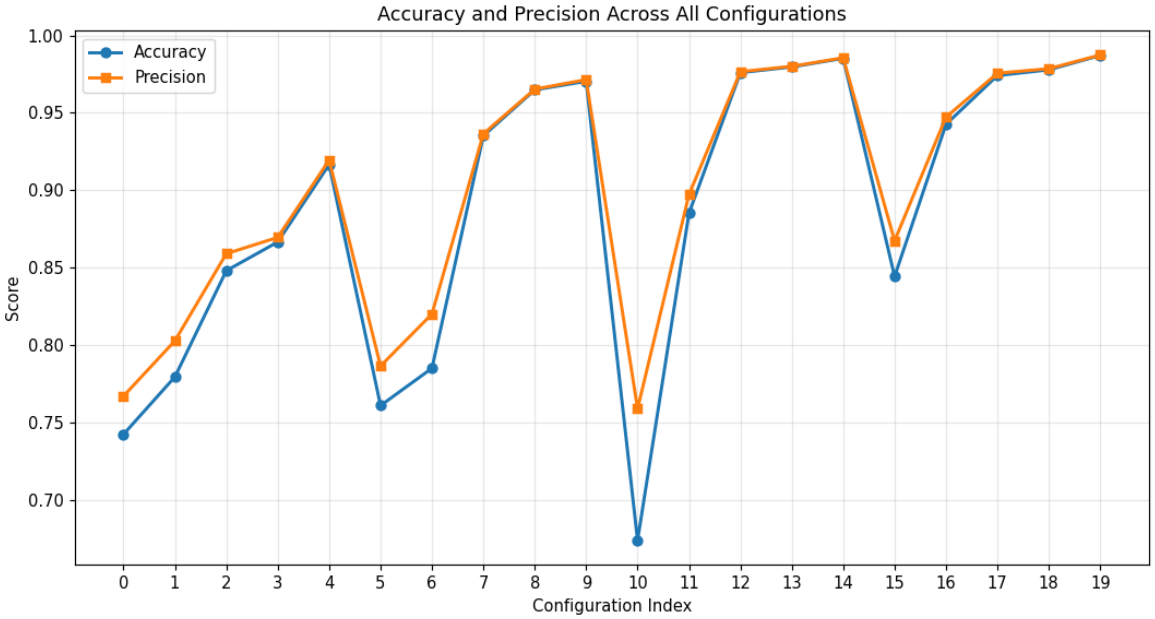
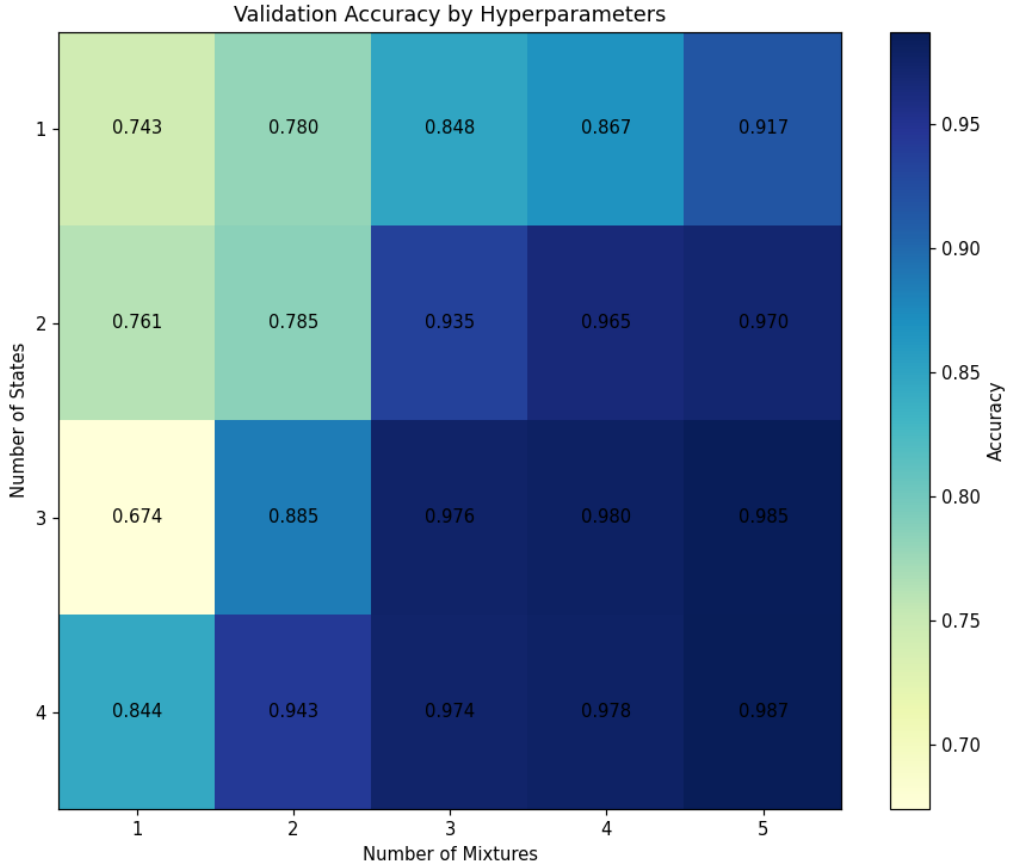


Figure 3: Accuracy και Precision κάθε διαμόρφωσης των GMM-HMM μοντέλων. Φαίνονται 4 φάσεις μήκους 5. Μεταξύ των φάσεων αυξάνεται το n_states, ενώ εντός κάθε φάσης αυξάνεται το n_mixtures.



Το καλύτερο μοντέλο είναι εμφανώς αυτό με 4 states και 5 gaussians ανά mixture και επιτυγχάνει accuracy = 98.7% στο validation set.

Τέλος, υπολογίζουμε την ακρίβεια του καλύτερου μοντέλου και στο testing set (ξανά καλώντας την evaluate αλλά με διαφορετικά δεδομένα). Παρατηρούμε ότι το μοντέλο έχει accuracy = 99% στο testing set.

```
Confusion matrix, without normalization
[[30  0  0  0  0  0  0  0  0  0]
 [ 0 30  0  0  0  0  0  0  0  0]
 [ 0  0 30  0  0  0  0  0  0  0]
 [ 0  0  1 29  0  0  0  0  0  0]
 [ 0  0  0  0 30  0  0  0  0  0]
 [ 0  0  0  0  0 29  0  1  0  0]
 [ 1  0  0  0  0  0 29  0  0  0]
 [ 0  0  0  0  0  0  0 30  0  0]
 [ 0  0  0  0  0  0  0  0 30  0]
 [ 0  0  0  0  0  0  0  0  0 30]]

Accuracy score on the test set is: 0.99
```

Σημειώνουμε ότι οι δοκιμές που κάνουμε στις παραμέτρους του μοντέλου, εκπαιδεύοντας μόνο στο training set θα έχουν κάποια ακρίβεια στο validation set. Με βάση μόνο αυτήν την ακρίβεια, επιλέγουμε το καλύτερο μοντέλο, και μόνο αφού έχουμε οριστικοποιήσει την επιλογή μοντέλου μας ελέγχουμε την απόδοση στο training set. Από το training set ελαχιστοποιούμε το σφάλμα εκπαίδευσης, από το validation set επιλέγουμε το καλύτερο μοντέλο, και από το testing set υπολογίζουμε εκτίμηση της ακρίβειας του μοντέλου που επιλέξαμε σε δεδομένα που δεν έχει ξαναδεί. Αυτό συμβαίνει, διότι αλλιώς, δηλαδή αν επιλέγαμε το καλύτερο μοντέλο με βάση την ακρίβεια στο training set, τότε ουσιαστικά θα εισάγαμε διαρροή δεδομένων από το testing set στο validation set, και η απόδοσή μας στο testing set δεν θα ήταν αντιπροσωπευτική της απόδοσής μας σε δεδομένα που το μοντέλο μας δεν θα είχε ξαναδεί. Το αντίστοιχο θα συνέβαινε αν χρησιμοποιούσαμε σε οποιοδήποτε σημείο προτού ολοκληρωθεί η εκπαίδευση του κάθε μοντέλο δεδομένα από το validation set. Για αυτόν τον λόγο, προσέξαμε πχ. νωρίτερα στον κώδικα να κάνουμε fit τον Scaler με βάση μόνο τα training data.

Βήμα 13

Τέλος, υπολογίζουμε τους πίνακες σύγχυσης του καλύτερου μοντέλου στο validation και στο testing set.

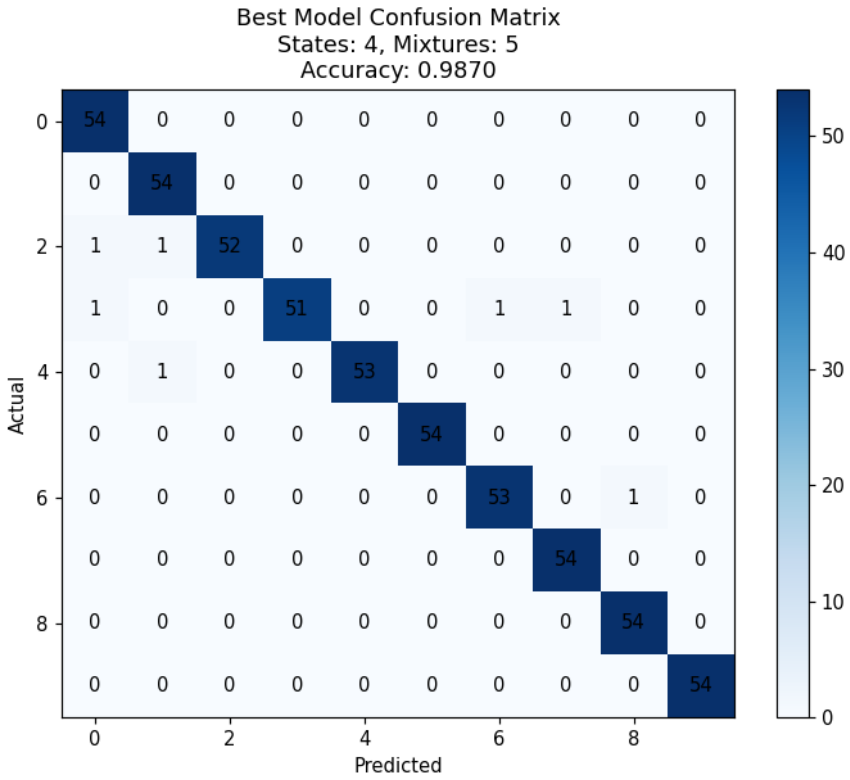


Figure 4: Πίνακας σύγχυσης του καλύτερου μοντέλου στο validation set

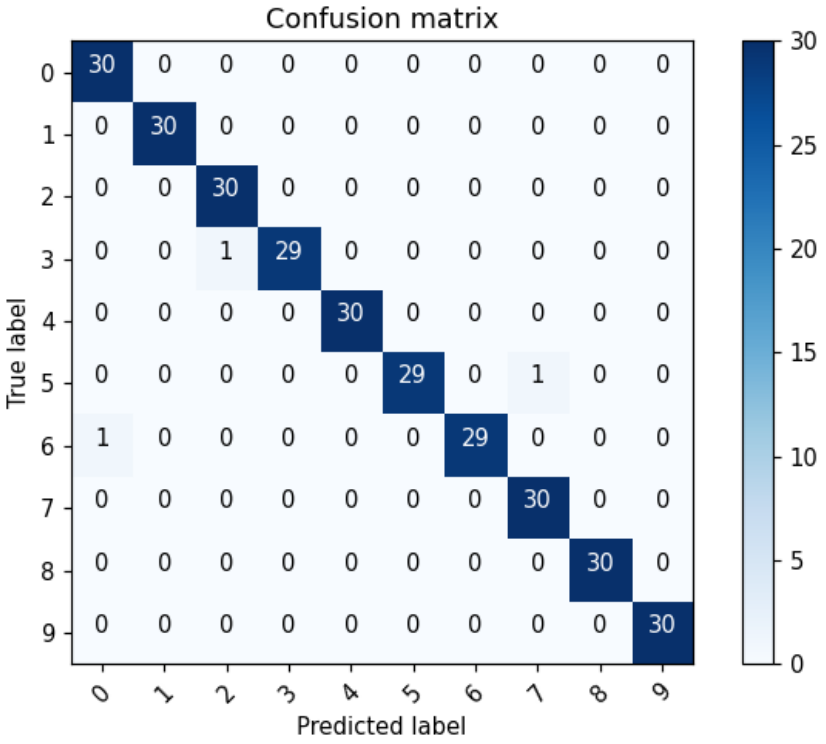


Figure 5: Πίνακας σύγχυσης στο testing set

Όπως βλέπουμε ξανά, η συνολική ακρίβεια του επιλεγμένου μοντέλου, δηλαδή το ολικό ποσοστό των σωστά κατηγοριοποιημένων εκφωνήσεων, στο testing set είναι 99%.

```
Confusion matrix, without normalization
[[30  0  0  0  0  0  0  0  0  0]
 [ 0 30  0  0  0  0  0  0  0  0]
 [ 0  0 30  0  0  0  0  0  0  0]
 [ 0  0  1 29  0  0  0  0  0  0]
 [ 0  0  0  0 30  0  0  0  0  0]
 [ 0  0  0  0  0 29  0  1  0  0]
 [ 1  0  0  0  0  0 29  0  0  0]
 [ 0  0  0  0  0  0  0 30  0  0]
 [ 0  0  0  0  0  0  0  0 30  0]
 [ 0  0  0  0  0  0  0  0  0 30]]

Accuracy score on the test set is: 0.99
```

Βήμα 14

1

Η υλοποίηση του βήματος αυτού βασίζεται στη δομή του βοηθητικού κώδικα για το εργαστήριο και συγκεκριμένα στο αρχείο *lstm.py*. Έχουν συμπληρωθεί τα τμήματα σημασμένα ως TODO και επιπλέον έχουν τροποποιηθεί λειτουργικότητες και προστεθεί επιπλεόν συναρτήσεις ώστε να καλυφθούν τα υποερωτήματα του βήματος.

2

Για την κατασκευή του LSTM δικτύου αξιοποιήσαμε τη βιβλιοθήκη PyTorch. Ο κώδικας φαίνεται παρακάτω:

```python
class BasicLSTM(nn.Module):
    def __init__(
        self,
        input_dim,
        rnn_size,
        output_dim,
        num_layers,
        bidirectional=False,
        dropout=dropout,
    ):
        super(BasicLSTM, self).__init__()
        self.bidirectional = bidirectional
        self.feature_size = rnn_size * 2 if self.bidirectional else rnn_size

        # TODO: YOUR CODE HERE
        # --------------- Insert your code here ---------------- #
        # Initialize the LSTM, Dropout, Output layers
        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=rnn_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0,
            bidirectional=bidirectional,
        )

        self.dropout = nn.Dropout(dropout)
        self.output_layer = nn.Linear(self.feature_size, output_dim)


    def forward(self, x, lengths):
        """
```

```
        x : 3D numpy array of dimension N x L x D
            N: batch index
            L: sequence index
            D: feature index

        lengths: N x 1
        """

        # TODO: YOUR CODE HERE
        # --------------- Insert your code here ---------------- #
        if not isinstance(x, torch.Tensor):
            x = torch.tensor(x, dtype=torch.float32)
        if not isinstance(lengths, torch.Tensor):
            lengths = torch.tensor(lengths, dtype=torch.int64)

        if use_packing:
            # sort the features in decreasing length for packing
            lengths_sorted, sort_idx = torch.sort(lengths, descending=True)
            x_sorted = x[sort_idx]
      # pack sequence to avoid processing the padding
            packed_input = pack_padded_sequence(x_sorted, lengths_sorted.cpu(),
     batch_first=True, enforce_sorted=True)
            # get packed output
            packed_output, _ = self.lstm(packed_input)
      # retrieve sequence in its original (padded) format
            outputs, _ = pad_packed_sequence(packed_output, batch_first=True)
            # retrieve the output in its original order
            _, unsort_idx = torch.sort(sort_idx)
            outputs = outputs[unsort_idx]
        else:
            outputs, _ = self.lstm(x)

        # You must have all of the outputs of the LSTM, but you need only the
    last one (that does not exceed the sequence length)
        # To get it use the last_timestep method
        # Then pass it through the remaining network
        last_outputs = self.last_timestep(outputs, lengths, self.bidirectional)

        last_outputs = self.dropout(last_outputs)
        last_outputs = self.output_layer(last_outputs)

        return last_outputs
```

Κατά την αρχικοποίηση του δικτύου ελέγχεται κατά πόσο η αρχιτεκτονική του είναι αμφίδρομη ή όχι. Εφόσον είναι, αφενός τίθεται σε True η αντίστοιχη παράμετρος και αφετέρου φροντίζουμε ώστε να διπλασιαστεί το μέγεθος του hidden state του δικτύου.

Στη συνέχεια, επισημαίνουμε τα είδη των layers που θα χρησιμοποιηθούν στην υλοποίηση του δικτύου. Το κυρίως layer είναι το nn.LSTM, το οποίο ρυθμίζεται με τις παραμέτρους του με τρόπο τέτοιο ώστε να καλύψει τις απαιτήσεις της εκφώνησης στα πιθανά ερωτήματα που θα προκύψουν. Το dropout layer δεν χρησιμοποιείται πάντα. Αυτό το καθορίζουμε με την παράμετρο dropout του nn.Dropout layer. Εάν αυτή είναι 0 προφανώς αυτή η τεχνική regularization δεν χρησιμοποιείται. Επιπλέον, αξίζει να αναφέρουμε ότι έχουμε φροντίσει το Dropout να μην εφαρμοστεί στο τελευταίο layer. Το nn.Linear layer χρησιμοποιείται ώστε να πάρει τα output του LSTM (για αυτό έχει και μέγεθος όσο του hidden state) και να τα συνδυάσει προκειμένου να πάρει την τελική απόφαση για το classification του ψηφίου.

Η συνάρτηση forward είναι αυτή που καθορίζει το πώς γίνεται το inference του μοντέλου. Δηλαδή, κατ' επέκταση, τη διάταξη των δομικών layers που ορίσαμε και εξηγήσαμε παραπάνω. Ο πρώτος διαχωρισμός που κάνουμε έγκειται στο αν χρησιμοποιούμε την τεχνική packing στο input (χρειάζε-

ται για το ερώτημα 8 - Bonus). Με την τεχνική του padding πετυχαίνουμε 2 πράγματα. Αρχικά, σε dataset με δεδομένα ακολουθίες διαφορετικών μηκών, είναι σκόπιμο να ενώσουμε τις μικρότερες από αυτές ώστε να τις επεξεργαζόμαστε ταυτόχρονα και να εξοικονομούμε χρόνο κατά την εκπαίδευση. Επιπρόσθετα, αποφεύγουμε το zero-padding και την επεξεργασία αυτής της παραμορφωμένης εκδοχής των ακολουθιών εισόδου. Η τεχνική του packing είναι ιδιαίτερα χρήσιμη σε δίκτυα στα οποία το μέγεθος εισόδου είναι αυστηρά καθορισμένο, όπως οι Transformers. Ο τρόπος που επιλέγουμε να εκτελέσουμε το packing είναι αρχικά να ταξινομήσουμε τις ακολουθίες με τα φωνήματα κατά φθίνον μήκος έτσι ώστε να τις συνδυάσουμε αποτελεσματικότερα. Αυτός ο συνδυασμός πραγματοποιείται από την συνάρτηση *packed_padded_sequence()* της βιβλιοθήκης της Pytorch (βλέπουμε ότι το enforce_sorted είναι ορισμένο σε True).

Τέλος, όπως βλέπει κανείς εδώ:

```
        last_outputs = self.last_timestep(outputs, lengths, self.bidirectional)

        last_outputs = self.dropout(last_outputs)
        last_outputs = self.output_layer(last_outputs)
```

```
    def last_timestep(self, outputs, lengths, bidirectional=False):
        """
        Returns the last output of the LSTM taking into account the zero padding
        """
        # TODO: READ THIS CODE AND UNDERSTAND WHAT IT DOES
        if bidirectional:
            forward, backward = self.split_directions(outputs)
            last_forward = self.last_by_index(forward, lengths)
            last_backward = backward[:, 0, :]
            # Concatenate and return - maybe add more functionalities like average
            return torch.cat((last_forward, last_backward), dim=-1)

        else:
            return self.last_by_index(outputs, lengths)
```

Έτσι εξασφαλίζουμε ότι ελέγχουμε χωριστά τις ξεχωριστές ακολουθίες (με χρήση των lengths) και ότι εφαρμόζουμε το dropout πριν από το τελευταίο layer.

3

Σε αυτό το βήμα καλούμαστε να εκπαιδεύσουμε το μοντέλο. Έτσι, αρχικά, θα εξηγήσουμε τη διαδικασία της εκπαίδευσής του.

```
def training_loop(model, train_dataloader, optimizer, criterion):
    model.train()
    running_loss = 0.0
    num_batches = 0
    for num_batch, batch in enumerate(train_dataloader):
        features, labels, lengths = batch
        # TODO: YOUR CODE HERE
        features = features.to(device).float()
        labels = labels.to(device)
        lengths = lengths.to(device)

        # zero grads in the optimizer
        optimizer.zero_grad()
        # run forward pass
        outputs = model(features, lengths)
        # calculate loss
        loss = criterion(outputs, labels)
```

```
18
19          # TODO: YOUR CODE HERE
20          # Run backward pass
21          loss.backward()
22          optimizer.step()
23
24          running_loss += loss.item()
25          num_batches += 1
26      train_loss = running_loss / num_batches
27      return train_loss
```

Στο training loop παίρνουμε τα δεδομένα του dataloader batch by batch, υπολογίζουμε το loss και κάνουμε το ανάλογο βήμα στον optimizer.

Ο optimizer που χρησιμοποιούμε είναι ο AdamW. Αξιοποιεί όρους ορμής και έχει μοντέλο για να μεταβάλει τους όρους του learning rate και της ταχύτητας κατάλληλα ώστε να επιτύχει την καλύτερη σύγκλιση. Προφανώς, εκτυπώνουμε το loss σε κάθε εποχή όπως μας ζητείται στην εκφώνηση.

4

Στην συνάρτηση train:

```
1       metrics = {
2           "training_loss": {},
3           "validation_loss": {},
4           "validation_accuracy": {},
5       }
6
7       confusion_matrices = {}
8
9       best_model_name, best_val_acc = None, -1
10
11      for model_name, model, optimizer in zip(models.keys(), models.values(),
        optimizers.values()):
12          model.to(device)
13
14          early_stopping = EarlyStopping(patience, mode="min", base_model=None)
15
16          print(f"Training in {device}")
17
18          metrics["training_loss"][model_name] = []
19          metrics["validation_loss"][model_name] = []
20          metrics["validation_accuracy"][model_name] = []
21          for epoch in range(epochs):
22              training_loss = training_loop(model, train_dataloader, optimizer,
        criterion)
23              valid_loss, y_pred, y_true = evaluation_loop(model, val_dataloader,
         criterion)
24
25              # TODO: Calculate and print accuracy score
26              valid_accuracy = accuracy_score(y_true, y_pred)
27              print(
28                  "Epoch {}: train loss = {}, valid loss = {}, valid acc = {}".
        format(
29                      epoch, training_loss, valid_loss, valid_accuracy
30                  )
31              )
32              metrics["training_loss"][model_name].append(training_loss)
33              metrics["validation_loss"][model_name].append(valid_loss)
34              metrics["validation_accuracy"][model_name].append(valid_accuracy)
35
```

```
            if valid_accuracy > best_val_acc:
                best_model_name = model_name

            if early_stopping.stop(valid_loss, model):
                print("early stopping...")
                model_weights = torch.load(checkpoint_filename, weights_only=
    False)
                break

    return best_model_name, metrics, models

def master_train(train_dataloader, val_dataloader, criterion):
    global use_early_stopping
```

Παίρνουμε τις μετρικές που μας ενδιαφέρουν, δηλαδή το training και validation loss καθώς και το validation accuracy. Η master train είναι αυτή που εκπαιδεύει όλα τα μοντέλα που μας ενδιαφέρουν.

5

Καλούμαστε να προσθέσουμε L2 Regularization στο μοντέλο μας, καθώς και dropout.

Το L2 regularization δεν είναι κάτι το οποίο επηρεάζει δομικά το μοντέλο μας άμεσα. Είναι ένας όρος ποινικοποίησης του μέτρου των βαρών που προστίθεται στο loss function. Έτσι, μειώνουμε την πολυπλοκότητα του μοντέλου και έχουμε ως αποτέλεσμα την αποφυγή του overfitting. Αυτό είναι σημαντικό και αναμένεται να παίξει ρόλο, ειδικά σε ένα απλό dataset όπως είναι το δικό μας. Ένα επιπλέον πράγμα που πετυχαίνουμε είναι να αποφύγουμε το saturation στα βάρη του δικτύου μας και να ενισχύσουμε έτσι τη δυνατότητα generalization του μοντέλου μας.

Η τεχνική dropout απενεργοποιεί ένα ποσοστό νευρώνων (καθορίζεται από την παράμετρο dropout) κατά τη διάρκεια της εκπαίδευσης έτσι ώστε να σιγουρευτούμε ότι όλοι θα εκπαιδευτούν και έτσι θα αποφύγουμε overfitting και το coadaptation. Προφανώς η απενεργοποίηση αυτή είναι προσωρινή και ο νευρώνας δυνητικά επανέρχεται στην επόμενη εποχή (απενεργοποιούνται άλλοι).

Τα χαρακτηριστικά που επιλέγουμε στην εκπαίδευση του μοντέλου μας είναι:

- epochs: 750
- learning rate: $2 * 10^{-5}$
- dropout: 0.4

Ο λόγος που επιλέγουμε μικρό learning rate είναι πως κατά τη διάρκεια της εκπαίδευσης παρατηρούσαμε ταλάντωση της μετρικής accuracy για μεγαλύτερα. Έτσι, οδηγηθήκαμε και στον σχετικά μεγάλο αριθμό εποχών (750), αφού λόγω του μικρού ρυθμού μάθησης το μοντέλο αργεί να συγκλίνει. Σταματήσαμε σε αυτόν τον αριθμό διότι από εκεί και πάνω δεν παρατηρούσαμε καμία βελτίωση.

6

Αρχικά, θα εξηγήσουμε τη σημασία του early stopping και κατόπιν θα προχωρήσουμε στην ανάλυση της δικής μας υλοποίησης.

Με το early stopping διακόπτουμε πρόωρα την εκπαίδευση του μοντέλου (πριν τη συμπλήρωση του προαπαιτούμενου αριθμού εποχών). Αυτό γίνεται με αυτόματο τρόπο. Ουσιαστικά, αν συμπληρωθεί ένας ορισμένος αριθμός εποχών (εδώ patience είναι 10) στον οποίο το μοντέλο μας δεν βελτιώνεται, σταματάμε. Με αυτό τον τρόπο πετυχαίνουμε τα εξής:

- Αποτρέπει το overfitting
- Εξοικονομεί χρόνο και πόρους. Αν το μοντέλο έχει πρακτικά συγκλίνει σε μικρότερο αριθμό εποχών από αυτόν που αναμένουμε, δεν συνεχίζουμε άσκοπα.

Στον κώδικά μας χρησιμοποιούμε την κλάση EarlyStopping:

```
class EarlyStopping(object):
    def __init__(self, patience, mode="min", base=None, base_model=None):
        self.best = base
        self.patience = patience
        self.patience_left = patience
        self.mode = mode
        self.pickle = None if base_model == None else torch.save(base_model.
    state_dict(), checkpoint_filename)

    def stop(self, value: float, model) -> bool:
        # TODO: YOUR CODE HERE
        # Decrease patience if the metric hs not improved
        if self.has_improved(value):
            self.best = value
            self.patience_left = self.patience  # reset patience
            self.model = model
            self.pickle = torch.save(model.state_dict(), checkpoint_filename)
        else:
            self.patience_left -= int(use_early_stopping)

        # Stop when patience reaches zero
        return self.patience_left <= 0

    def has_improved(self, value: float) -> bool:
        # TODO: YOUR CODE HERE
        # Check if the metric has improved
        if self.best is None:
            return True

        if self.mode == "min":
            return value < self.best
        elif self.mode == "max":
            return value > self.best
        else:
            raise ValueError(f"Invalid mode '{self.mode}'. Use 'min' or 'max'."
    )
```

Κεντρική μέθοδός της είναι η stop. Ουσιαστικά ελέγχει κατά πόσο έχει βελτιωθεί το μοντέλο μας σε σχέση με την προηγούμενη εποχή (το συγκρίνει με το μέχρι τώρα καλύτερο) και, εφόσον δεν έχει βελτιωθεί, αφαιρεί μία εποχή από το περιθώριο υπομονής μας (*patience_left*). Όταν αυτό εξαντληθεί, δηλαδή φτάσει στο 0, αποθηκεύουμε το μοντέλο μας σε pickle format με τις κατάλληλες συναρτήσεις της PyTorch.

7

Σε αυτό το υποερώτημα καλούμαστε να πειραματιστούμε και με ένα bidirectional LSTM. Όπως έχουμε εξηγήσει παραπάνω, ο τρόπος ώστε το μοντέλο μας να γίνεται αμφίδρομο είναι να θέσουμε True την κατάλληλη παράμετρο και οι κατάλληλες προσαρμογές γίνονται αυτόματα από τον κώδικά μας για το μοντέλο.

Ουστιαστικά, το bidirectional LSTM έχει τα διπλάσια κύτταρα LSTM από το απλό. Τα μισά διαδίδουν πληροφορία του hidden state προς τα εμπρός, όπως και πριν, ενώ τα άλλα μισά κάνουν το ίδιο αλλά στην αντίστροφη κατεύθυνση.

Η ανάγνωση της ακολουθίας εισόδου τόσο από την αρχή προς το τέλος της όσο και αντίστροφα, μας επιτρέπει να καταλάβουμε το context και τα συμφραζόμενα καλύτερα. Προφανώς, αυτό ισχύει και έχει σημασία και στην προφορά των ψηφίων αφού για παράδειγμα τόσο το two όσο και το three ξεκινούν με το ίδιο σύμφωνο ήχο ενώ τελειώνουν με διαφορετικό. Επίσης, με τα αμφίδρομα LSTM μπορούμε να αντιμετωπίσουμε καλύτερα την "απώλεια μνήμης" για μεγάλες ακολουθίες. Προφανώς όταν μία ακολουθία είναι μεγάλη, το context και η πληροφορία της αρχής ξεθωριάζει καθώς ξεδιπλώνουμε το LSTM. Τώρα, όμως, διαβάζοντάς τη και από το τέλος έχουμε καλύτερη συγκράτηση και συνδυασμό πληροφορίας από όλο το μήκος της ακολουθίας. Η αύξηση αυτή της χωρητικότητας του μοντέλου που περιγράφουμε οδηγεί συνήθως και σε αύξηση της απόδοσής του.

8

Στο τελευταίο Bonus υποερώτημα καλούμαστε να πειραματιστούμε με την τεχνική του packing. Ο τρόπος με τον οποίο λειτουργεί εξηγήθηκε στο υποερώτημα 2. Συνοψίζουμε τα πλεονεκτήματά του:

- Αποφυγή θορύβου από το padding στο dataset
- Αύξηση της ταχύτητας εκπαίδευσης αφού βλέπουμε πολλές μικρές ακολουθίες ταυτόχρονα αντί ξεχωριστά
- Βελτίωση της αριθμητικής ευρωστείας του μοντέλου, αφού όλο και λιγότερα δείγματα (συνεχόμενα κιόλας) είναι μηδενικά και έτσι τα gradients είναι "καθαρότερα"

Αποτελέσματα

Μετά την περιήγηση στα υποερωτήματα μας θα παρουσιάσουμε τα αποτελέσματα των πειραματισμών μας.

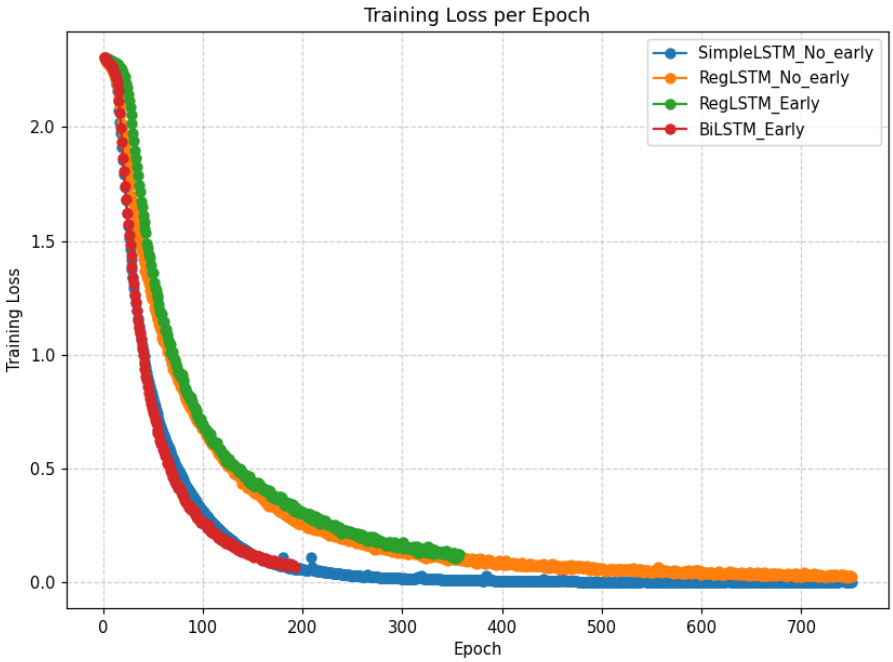


Figure 6: Models' Train Loss

Το να παραθέσουμε τα prints του training loss σε κάθε εποχή μπορεί να βοήθησε στο tuning των epochs και του learning rate αλλά δεν έχει νόημα να τα παρουσιάσουμε με λεπτομέρεια στην

αναφορά. Αντί αυτού έχουμε κάνει το παραπάνω plot που απεικονίζει τη σύγκλιση όλων των μοντέλων που εκπαιδεύσαμε.

Όπως παρατηρούμε στο διάγραμμα, το early stopping πράγματι λειτουργεί διακόπτοντας την εκπαίδευση των μοντέλων που το χρησιμοποιούν όταν αυτά έχουν σχεδόν σταθεροποιηθεί. Αυτό γλιτώνει πάνω από το μισό χρόνο εκπαίδευσης δίχως να έχει μεγάλη επίπτωση στην απόδοση - όπως θα δούμε σε λίγο. Επίσης, όπως είναι λογικό, τα μοντέλα που χρησιμοποιούν regularization γενικά συγκλίνουν πιο αργά. Αυτό συμβαίνει αφενός λόγω του dropout το οποίο απενεργοποιεί διάφορους νευρώνες καθιστώντας πιο αργή την individual σύγκλισή τους και αφετέρου επειδή το L2 regularization δεν ευνοεί επίσης την ταχύτητα σύγκλισης. Το Bidirectional LSTM χρησιμοποιεί και αυτό regularization αλλά συγκλίνει πολύ γρήγορα λόγω της ευκρίνειας στα gradients που δίνεται από την πιο πλούσια πληροφορία.

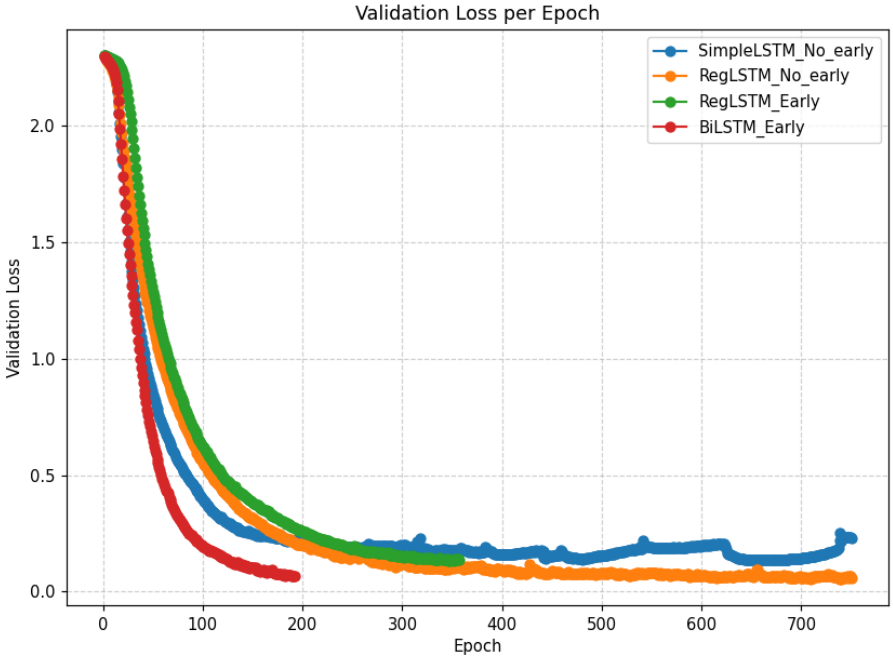


Figure 7: Models' Validation Loss

Γενικά, ένα μοντέλο στο οποίο δεν υπάρχει overfitting, πρέπει να έχει παρόμοια εξέλιξη στην σύγκλιση του validation και του trainning accuracy και πιο σημαντικά, στην κατάσταση σύγκλισης να μην υπάρχει χάσμα μεταξύ στις δύο μετρικές (προφανώς δεν μπορεί ποτέ να έχουμε καλύτερο accuracy στο validation set από ότι στο train αλλά πρέπει να είναι κοντά). Όπως βλέπουμε στο διάγραμμα, στο μόνο μοντέλο στο οποίο αυτό δεν συμβαίνει, δηλαδή βλέπουμε δείγματα overfitting είναι στο απλό μοντέλο δίχως χρήση regularization. Αυτό είναι απολύτως λογικό, αφού η χωρητικότητα του μοντέλου υπερκαλύπτει τις ανάγκες του task και δεν υπάρχει κανένας μηχανισμός αποφυγής της υπερπροσαρμογής του στα δεδομένα εκπαίδευσης. Αυτό δεν συμβαίνει με τα υπόλοιπα μοντέλα που χρησιμοποιείται regularization πράγμα που επιβεβαιώνει την αποτελεσματικότητά του και τη σωστή χρήση του στον κώδικά μας.

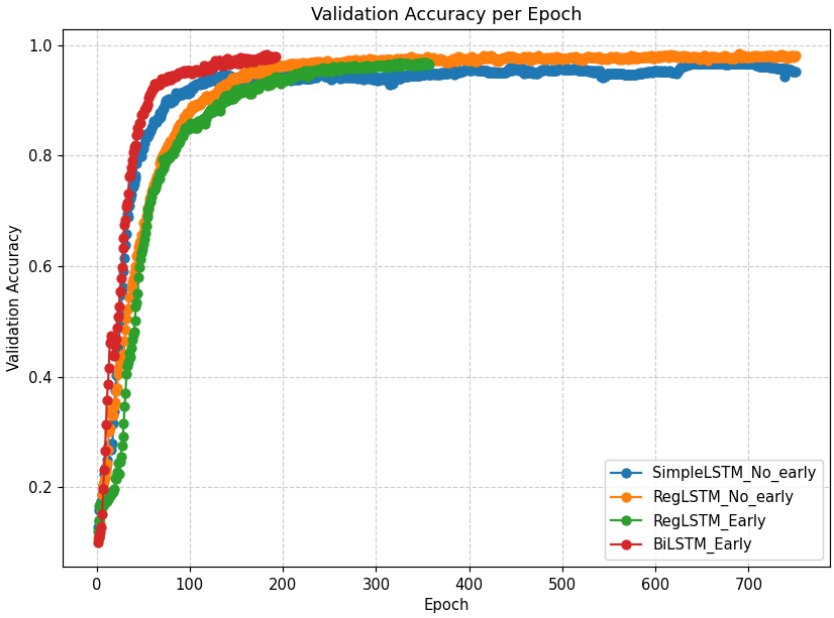


Figure 8: Models' Validation Accuracy

Όπως είναι λογικό, το accuracy στο validation set ακολουθεί αντίστροφη πορεία σύγκλισης από το loss σε όλα τα μοντέλα.

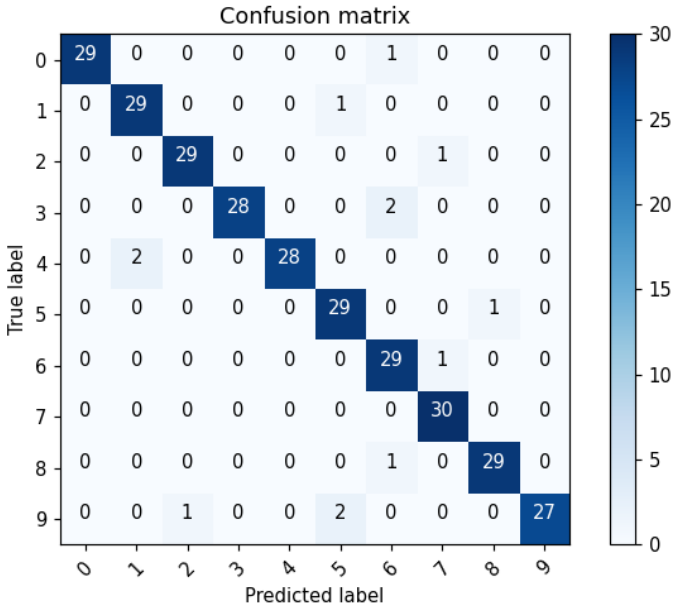


Figure 9: Confusion Matrix

Το confusion matrix στο test set δείχνει πως το καλύτερο μοντέλο μας τα πάει εξαιρετικά αφού κάνει ελάχιστα λάθη σε όλα τα ψηφία.

```
Model training time without packing: 82.526691198349.
Model training time with packing: 288.780015707016.
The best model was SimpleLSTM_No_early
```

Figure 10: Confusion Matrix